Software Engineering

NECKER

September 26, 2026

Contents

1 Engineering, Quality, and Processes

History

- 1950s-1960s: need to move beyond "craftsmanship" methods of software development.
- Main problems: communication between client and programmer, application domain no longer exclusively mathematical.
- 1970s: emergence of methods, processes, and tools to improve software quality, engineering approach.

Engineering Approach

- Target: goal to be achieved.
- Metric: metric to measure the achievement of the target.
- Method/Process/Tool: techniques to approach the goal.
- Measurements: controlled experiments to evaluate effectiveness:
  - Satisfactory results $\Rightarrow$ accept methods/processes.
  - Unsatisfactory results $\Rightarrow$ revise target, metric, processes, or methods.
- Types of goals:
  - Solve development problems.
  - Ensure software quality.

Main Problems

- Communication:
  - Developer $\leftrightarrow$ Client: different domains, multiple people involved.
  - Developer $\leftrightarrow$ Developer: different styles and specializations.
- Software Size: millions of lines of code, thousands of man-years (if an employee works 8 hours a day, then 8 hours = one man-day).
- Software Malleability: versions, evolutions, changes in targets.
- Man-years measure: not indicative, non-linear with the number of developers.

Quality

qualities we want the software to have

- Types:
  - External: perceived by clients.
  - Internal: perceived by developers, also influence external qualities.
- Requirements vs Specifications:
  - Requirements: what the client wants, changeable.
  - Specifications: developer's interpretation of requirements, contractual basis.

Software Quality

- Correctness: software adheres to specifications.
- Reliability: the system can be trusted to perform according to requirements.
- Robustness/Safety: acceptable behavior even outside specifications.
- Usability: easy and intuitive for users, qualitative and quantitative testing.
- Performance/Efficiency: external performance vs internal resource consumption.
- Verifiability: easily verifiable properties, related to readability or formal methods.
- Reusability: reusable components, increases reliability and verifiability.
- Maintainability:
  - Repairability: defect correction.
  - Evolvability: addition of new functionalities.
- Perfectibility: improving quality without changing functionality.

Relevant Laws

- Glass (G1): errors in requirements ⇒ project failure.
- Nielsen-Norman (NN23): usability is measurable.
- McIlroy (MI15): software reuse increases productivity and quality.
- Lehman (L27-28): systems evolve, complexity increases if not managed.

Technical Debt

- Postponing fixes creates "technical debt".
- Debt generates interest: increased difficulty and development time for new features due to bad code.

Process Quality

having discussed the desired software qualities, we must now introduce process qualities, i.e., how we do it; a good process must have:

- Robustness: resisting unexpected events (e.g., sudden staff shortage).
- Productivity: (bear in mind that a team's productivity is less than the summed productivity of individuals.)
- Timeliness:
  - Meeting deadlines.
  - Managing requirements volatility.
  - Grasping the market context.

Software Production Process

- Problems:
  - Requirements change.
  - Software $\neq$ just writing code.
  - Complex communication.
  - Need for rigor (Bauer-Zemanek Hypothesis: Formal methods significantly reduce design errors, or allow them to be eliminated and resolved earlier.).

- Activity organization:
  - Identify activities.
  - Define precedence and responsibilities.
- Initially, the code-and-fix model was used, which was highly ineffective.

Lifecycle Phases

- **Feasibility study**: scenarios, architectures, costs, ROI (return on investment).
- **Requirements analysis and specification**:
  - Understand domain, dialogue with client, identify stakeholders.
  - Define functionality (what, not how).
  - Common dictionary, non-functional requirements (requires extreme precision).
  - Analysis output: specification document, test plan, user manual. The document can be: descriptive or operational (David's Law: the value of models representing software from different perspectives depends on the chosen viewpoint (assumption), but no single view is best for every purpose.)

  descriptive vs operational model

  - descriptive: a model describing how it is made, more like specific documentation.
  - operational: a sort of prototype, more focused on: it must be roughly like this.
- **Design**: the second step is planning and design:
  - Software architecture.
  - Decomposition into modules/objects.
  - Pattern identification.
- **Programming and unit testing**: module development, unit tests, stubs, and drivers.

- **Integration and system testing**: module integration, incremental testing, alpha testing.
- **Delivery, installation, and maintenance**: beta testing, deployment, corrective/adaptive/perfective maintenance.
- **Other activities**:
  - Documentation.
  - Verification and quality control.
  - Process and configuration management.

2 Software Lifecycle Models

No universally valid model exists.
Types of models:

- **Prescriptive:** impose precise rules.
- **Descriptive:** describe existing processes without imposing them.

2.1 Sequential Models

2.1.1 Waterfall Model

Characteristics:

- Sequential phases: Requirements → Design → Coding → Testing → Product.
- Each phase produces a semi-finished product (document-based); once moved to the next phase, the work for the previous phase is considered finished.
- Separation of duties, rigor, and simple temporal planning.

Pros:

- Document-based.
- Good division of tasks.
- Clear planning.

Cons:

- Rigidity, impossibility to go back.
- Freezing of sub-products.
- Monolithic nature, absence of real maintenance.

Historical Note: Not praised, but used as a negative comparison by Royce (1970), who criticized its limitations while proposing incremental approaches.

2.1.2 V-Model (Shark Teeth)

Idea: Variant of the waterfall model featuring phases of: *verification* and *validation*.

Verification: formal evaluation against specifications (no client).
Validation: evaluation against client needs with continuous feedback.

Objective: Involve the client and ensure consistency at every stage.

2.2 Iterative Models

General Idea: allow the repetition of phases to increase flexibility and adaptability to changes.

2.2.1 Waterfall Model with Single Feedback Loop

- Allows jumping back by a single phase (e.g., from Testing to Coding).
- In reality, we notice that we can perform complete iterations up to delivery, given that having stepped back by one phase, we can evaluate stepping back further.
- Difficult to plan timelines and progress.

2.2.2 Prototyping Model

Characteristics:

- Introduces throw-away prototypes.
- Purpose: receive feedback or test ideas/tools.
- **Public prototypes:** to clarify client requirements.
- **Private prototypes (spikes):** exploratory, to better understand the problem.

Boehm's Law (L3): Prototyping significantly reduces requirements analysis and design errors, especially for user interfaces. **Iterative nature:** each cycle produces and discards a prototype until the final version is reached.

2.3 Incremental Models

Definition: iterative models where each iteration also includes a delivery.
Differences:

- **Iterative:** iterations on development.
- **Incremental:** iterations across all phases, including specifications and release.

Advantages: natural integration of maintenance and evolutionary software management.

2.3.1 Fountain Model

- Origin: 1993, opposed to the waterfall model.
- One can always return to the initial phase (software pool: collection of reusable code parts) to fix errors.
- Introduces post-delivery maintenance and evolution.
- Loss of linearity and impossibility of estimating timelines.

2.3.2 Common Problems of Incremental Models

- Complex planning, progress status is less visible.
- Need for experts to be constantly available.
- Potential non-convergence of the project.
- Overlapping between iterations (hence two versions produced at the same time) $\rightarrow$ little time for feedback.

2.3.3 Pinball Life-Cycle

- Model created as an ironic critique by Ambler: random and uncontrollable order of activities.
- Non-measurable and chaotic process (pessimistic but realistic in unstructured contexts).

2.4 Transformational Models

- Opposite of Pinball: maximum formality and control.
- Starts from informal requirements, applying formal transformations down to the final code.
- Each step produces a formally correct prototype.
- Introduces formal proofs of correctness and versioning via transformations.

Applications: due to their extreme rigidity, they are mainly used in research environments or hardware/software co-design projects (e.g., processors).

2.5 Spiral Metamodel

Characteristics:

- Risk-based incremental metamodel; note that it differs from a standard incremental model in this way.
- Phases:
  1. Determination of objectives, alternatives, and constraints.
  2. Evaluation of alternatives and risks.
  3. Development and verification.
  4. Planning the next iteration.
- Radius of the spiral = increasing costs.

Win-Win Variant: considers contractual risks, searching for client-developer balance; indeed, at each iteration, a compromise is sought between client and developers.

2.5.1 COTS Model (Component Off The Shelf)

Idea: development based on pre-existing components (internal or commercial). **Phases:**

1. Requirements analysis.
2. Analysis of available components.
3. Modification of requirements (if necessary).

4. System design incorporating reuse.
5. Development and integration.
6. System verification.

Pros: reuse, speed.
Cons: integration complexity, inefficiency due to unused features.

2.6 Agile Methodologies

Idea: PRESCRIPTIVE methodologies born "from the bottom up" to address the limitations of traditional methods.

2.6.1 Agile Manifesto

> implies: more important than

- Individuals and interactions > processes and tools.
- Working software > comprehensive documentation.
- Customer collaboration > contract negotiation.
- Responding to change > following a plan.

Business model: time-and-materials payment structure with direct client collaboration.

2.6.2 Lean Software

- Inspired by Toyota's *Lean Manufacturing*.
- Objective: reduce waste, avoid non-deliverable products (efforts are made to avoid creating demos or prototypes that would only waste time since they would eventually be discarded).
- Reuse code, parallelize processes, postpone binding choices.
- Invest in developer well-being.

2.6.3 Kanban

- Minimize *work in progress* (focus on one task and avoid parallel work).
- Tables with 5 columns: backlog, to do, in progress, in testing, done.
- Each card is assigned to a developer or a pair.
- If a member is blocked, they help others instead of starting new work.

2.6.4 Scrum

- Short iterations (2-4 weeks).
- Fixed requirements during the iteration (to keep the developer focused on a single piece of work).
- Client involved only between iterations.
- Focus and stability for the team.

2.6.5 Crystal

- Introduces **osmotic communication**: shared knowledge among members; every phase is managed by everyone, no single expert is solely responsible for an individual phase.
- Collective code ownership → robustness.
- Effective for small teams (fewer than 10 members).
- for large teams, evolutions exist, such as the "SAFe" framework.

3 Extreme Programming (XP)

also an agile methodology

3.1 Test Driven Development (TDD)

- Software **design** technique that drives out the simplest design possible.
- It is neither verification nor mere code writing, but an **approach to writing**.
- Formula: **TDD = test-first + baby steps**.

- Objective: **rapid and continuous feedback**.
- Main cycle: **Red → Green → Refactoring**.
  - **Red**: write the test before the code → it fails (feature does not exist yet).
  - **Green**: write the minimum code to make the test pass.
  - **Refactoring**: improve the code while keeping the tests green.
- Cycle repeated every 2–10 minutes ⇒ focus on small tasks.
- Advantages:
  - Designs **testable and simple** code.
  - Defines clear interfaces and reduces dependencies.

3.2 Foundations of XP

- Created by **Kent Beck** (late 1990s, Chrysler project).
- Fundamental variables (interdependent on each other):
  - **Scope**: functionalities to be implemented (primary variable).
  - **Time**: available temporal resources.
  - **Quality**: must remain constant (the software must work).
  - **Cost**: economic/staff resources employed.
- In XP: quality is constant, cost and time are balanced → **scope is adaptive** (making scope adaptive allows holding the other variables constant).
- Incremental approach and continuous delivery:
  - Client is part of the team and receives **incremental releases**.
  - Can modify requirements and priorities along the way.
- Result: a more **collaborative and transparent** relationship between client and developer.

3.3 XP Principles

- Adopts classical principles:
  - separation of concerns: separating tasks.
  - abstraction and modularity: using the right abstraction, right language, etc.
  - Anticipation of change: the project may evolve in the future, prevention is better than cure.
  - generality: more general interfaces are better to cope with possible changes.
  - rigor: rigidity and precision in specifications.
  - incrementality: not everything at once but a solid development a little at a time.
- Introduces new principles, inheriting only some classical ones (separation of concerns is taken for granted):
  - **Rapid feedback**: continuous feedback from tests, colleagues, and client (e.g., standup meetings).
  - **Assume simplicity**: avoid useless complexity in code and roles.
  - **Embrace change**: anticipate modifications but without designing for them in advance.
  - **Incremental change**: short iterations and small functionalities.
  - **Quality work**: focus on developer well-being.
- Key contrast: **Simplicity vs Anticipation of change**.
  - XP: simple code is better, as it is easily modifiable.
  - Cost curves:
    - **Boehm (1976)**: exponential modification cost; according to him, failing to plan implies greater difficulty in the future.
    - **XP**: according to XP, the cost is logarithmic thanks to refactoring and modern languages.

3.4 Roles and Responsibilities

- Main figures:
  - **Client**: knows the domain, defines functionality and priorities.

  - **Developer**: implements, estimates timelines, and defines technical details.
  - **Manager**: supervises, plans, and monitors.
  - **Tracker**: monitors team issues and metrics (e.g., open issues).
- Key concepts:
  - **Business value**: measures the importance of a feature relative to its cost.
  - Transparency and trust between client and developer.

3.5 XP Techniques (13 Core Techniques)

1. **Planning Game**: initial planning of each iteration.
2. **Short release cycles**: releases every $\sim$2 weeks.
3. **Use of a metaphor**: common vocabulary for developers and client.
4. **Assume simplicity**: clear and direct code.
5. **Testing**: continuous and automated tests.
6. **Refactoring**: continuous improvement of code.
7. **Pair Programming**: two developers at one workstation.
8. **Collective ownership**: code belongs to the entire team.
9. **Continuous integration**: frequent builds and tests.
10. **40-hour week**: avoid overload.
11. **On-site customer**: direct and daily collaboration.
12. **Coding standards**: uniformity in style.
13. **They're just rules**: flexibility in application.

3.6 Planning Game — Details

- Purpose: determine which features to implement in the next iteration.
- Tool: **user stories** written on cards (subject, action, motivation).
- Cards contain: ID, description, acceptance test, business value.
- Phases:
  1. The client writes the cards (user stories).
  2. The team estimates time (in hours, days, or story points).
  3. The manager selects the cards to be built based on value and time.
- Estimation problems:
  - Highly divergent estimates → ambiguous card.
  - High estimates → story too large (requires **splitting**).
  - **Anchoring** effect: the first estimate influences subsequent ones.

3.6.1 Estimation Techniques

- **Planning Poker**:
  - Each member chooses a face-down card (numbers from 0 to 100, non-linear scale).
  - Cards are revealed simultaneously.
  - Those with extreme values explain their choice.
  - Repeated until unanimity is reached.
- Special cards:
  - **?** = cannot estimate.
  - **Coffee cup** = coffee break.
- **Team Estimation Game** (resolves the anchoring problem):
  1. **Phase 1**: qualitative sorting (easier/harder); one developer at a time places a card to the left or right relative to others to indicate if it takes less or more time.
  2. **Phase 2**: numerical value assignment (using planning poker cards); used to compress the row from phase 1 by assigning a number and grouping user stories of similar difficulty.
  3. **Phase 3**: eventual estimation in man-hours (often useless).

3.6.2 Velocity

- Definition: **story points completed per iteration**.
- Non-comparable across different teams (mainly serves to tell the team how many tasks to take on based on the performance of the previous iteration).
- Must not include incomplete stories.
- Can vary with the addition or removal of members (adding a member should not influence it initially, as they must be trained first).
- **No estimates** movement: zero estimates, relying solely on experience and uniformity in stories (a variant that works only in highly mature teams).

3.7 Short Release Cycles

- Frequent releases (every $\sim$2 weeks).
- Each release must be **working**.
- Client must have time to evaluate and modify requests at the end of each iteration.
- Bertrand Meyer says: "*Brilliant idea, perhaps the most influential in the agile industry*".

3.8 Use of a Metaphor

- Defines a **common vocabulary** between client and developers.
- Replaces documentation as a system overview.
- Facilitates understanding of the system and discovery of new features.
- Difficult to create in practice.

3.9 Simplicity of Design

The art of **maximizing the amount of work not done**.

- XP Slogans:

  - **KISS**: Keep It Simple, Stupid.
  - **Once and only once**: zero duplication.
- Opposed to *design for change*, considered an unnecessary burden.
- Better to ask the client what is needed right now rather than predicting.
- Simplicity does not necessarily imply lower efficiency.

3.10 Testing

- Two types:
  - **Acceptance/functional tests**: written by clients within *user stories*.
  - **Unit tests**: written by programmers before writing code.
- Intense testing: acts as a safety net across all phases.
- Every change is verifiable through tests.
- Fundamental also in refactoring and maintaining legacy code.
- Tests drive development (**TDD**, Test-Driven Development); by writing the test before the code, one has a much clearer idea of what needs to be done.
- Tests must cover as many lines of code as possible.

3.11 Refactoring

- Modifies the **internal properties** of the software, not its behavior.
- Objective: reduce entropy, increase readability and simplicity.
- Must be continuous and gradual.
- Modifications must always be driven by tests.
- Essential to ensure code evolvability and quality.
- Courage is required, even for novices, when modifying code.
- Restructuring legacy systems (very large changes) may require different strategies.

3.12 Pair Programming

- Two developers at a single workstation: **driver** and **navigator**.
- Advantages:
  - Mutual review of code and adherence to XP rules.
  - Better refactoring and continuous sharing of ideas.
  - Facilitates onboarding of new members (osmotic learning).
  - Increases **collective ownership** of code.
  - "Naive" comments help clarify concepts taken for granted.
- Productivity increases by ∼50% (meaning: if one person has a productivity of 100, two together have 150), but overall quality improves significantly.
- Reduces verification and validation times.

Critiques by Bertrand Meyer

- Pair programming is useful, but should not be imposed as the sole method.
- Conclusive empirical evidence of universal benefits is lacking, making it more of an ideology.
- People are different: some programmers work better in solitude.
- Forcing a single working model is risky in a creative activity.

3.13 Collective Code Ownership

- Code belongs to the entire team, not to individuals.
- No "code ownership" policies.
- Everyone can modify any part of the code.
- Pairs change multiple times a day.
- Promotes distributed knowledge and collaboration.

3.14 Continuous Integration

- Integrate frequently (multiple times a day).
- Integration must be gradual and always **ready for delivery**.
- Each completed feature must be integrated, tested, and shown to the client.
- Avoids conflicts between pairs and accumulation of problems.
- Each pair integrates on a shared machine with exclusive access (token or similar).
- If a pair fails to integrate, they must back out and resolve issues locally.
- All pairs have a personal test environment.

3.15 40-Hour Week

- Avoid overtime and prolonged stress.
- Objectives:
  - Higher productivity and satisfaction.
  - Better work-life balance.
  - Reduction of turnover and burnout.
- Intellectual work is not mechanical: one still tends to think about problems outside working hours.

3.16 On-Site Customer

- Client physically present at the development location.
- Advantages:
  - Greater control for the client.
  - Possibility to clarify specifications immediately.
  - Client acts as **living documentation**.
- Must be representative of all stakeholders.
- If unavailable: introduce a **Product Owner** (a team member who acts as the client, as in Scrum).
- Client can create new *user stories* for subsequent iterations.

3.17 Coding Standards

- Define common code writing conventions.
- Increases readability and communication through code.
- Promotes:
  - Refactoring.
  - Pair programming.
  - Collective ownership.
- Automatic tools can verify or correct style.

3.18 They're just rules

- Rule added later by some agilists.
- At the end of each iteration, the team can decide:
  - How to behave in the next one.
  - Which rules to suspend or reactivate.
- Decision always at the **team** level, never individual.
- XP is not rigid: adaptable to the group's needs.
- However, practices are highly interconnected and should not be viewed in isolation.

Grouping into Phases

Requirements

- Clients are part of the team: *living requirements*.
- Incremental deliveries and continuous planning: project evolution.

Design

- Metaphor as a unifying vision of the project.
- Refactoring: pure design, enables software evolvability.
- Assume simplicity.

Code

- Pair programming.
- Collective code ownership.
- Continuous integration.
- Coding standards.

Test

- Continuous unit testing, written before the code.
- Functional tests written by users within *user stories*.

Spike

- Simple prototypes to explore solutions.
- Help understand specifications, technology, and interaction with external components.
- Created, shown to the client, and finally discarded.

Documentation

- Paper documentation is not necessary (on-site customer, stories, peer programming companion...).
- Documentation replaced by:
  - Unit tests (executable specifications, written before code).
  - Continuous refactoring to maintain readable code, eliminating comments.

CRC Cards

- Class Responsibility and Collaboration cards represent classes and relationships.
- Usage similar to the *planning game*, but from the programmer's perspective.
- Content of each card:
  - Class name
  - Superclasses and subclasses (if applicable)
  - Responsibilities
  - Collaborating classes
  - Author
- Small sizes to limit complexity and responsibility and to allow gestural activities (moving, reorganizing, discarding cards).
- Placement of cards can indicate collaboration or supervision relationships.

Criticalities: When Not to Use XP

- Agile is not suitable if:
  - Distributed teams.
  - Teams that are too large.
  - Technological barriers (e.g., impossibility of continuous integration).
  - Too many conflicting stakeholders.
  - Incremental delivery makes no sense (e.g., nuclear power plants).

Critiques from Meyer

- Underestimation of *up-front* design (unless one is an expert, designing from the bottom up is highly complex).
- Overestimation of *user stories*.

- Failure to highlight dependencies between *user stories* (they should be independent, but it is often impossible).
- TDD can lead to a narrow vision.
- Problems in *cross-functional* teams that are too heterogeneous (many individual specialized figures who must collaborate in pairs).

Man-Months

often an incorrect estimate for calculating development time

- Estimation in man-months does not consider communication overhead.
- Increasing the number of people does not linearly increase productivity.
- In sequential or non-parallelizable work, more people can slow things down.
- Solutions in case of delays:
  - Discuss the *deadline* with the client.
  - Decrease the scope of the project.
  - Decrease quality (reduced *testing*, not recommended).
- Ideal: team of 8-10 people; beyond this, agile methodologies may fail.

Open Source

Definition

- Software whose **source code is freely viewable online**.
- Development based on a **community of independent developers**, not a centralized team.

Reference Literature

- *The Cathedral and the Bazaar* – Eric S. Raymond
- *Care and Feeding of FOSS* – Craig James
- *The Emerging Economic Paradigm of Open Source* – Bruce Perens
- *An empirical study of open-source and closed-source software products* – J.W. Paulson

The Cathedral and the Bazaar (Raymond)

Two Metaphors:

- **Cathedral (closed source)**: hierarchical structure, central control, a single architect.
- **Bazaar (open source)**: anarchic environment, spontaneous cooperation, continuous expansion.

Birth of an Open-Source Project

1. A developer encounters a problem and creates an application to solve it.
2. Involves friends/colleagues with the same problem.
3. A small informal group forms, initiating development and exchanging knowledge.
4. A first working version is obtained.
5. The project becomes **open-source** when the code is made public.
6. The community provides feedback, patches, and new skills.
7. A cycle of continuous improvement is established.

Key Concepts

- "*Every good software work starts by scratching a developer's personal itch.*"
- "*If you give everyone the source code, every one of them becomes your engineer.*"
- "*Given enough eyeballs, all bugs are shallow.*"
- "*Treat your beta-testers as your most important resource.*"
- "*When you lose interest in a program, your last duty is to hand it off to a competent successor.*" (often not done, and the project dies)

Comparison of Development Models

Aspect	**Traditional**	**Agile**	**Open Source**
Documentation	Quality control tool	De-emphasized	All artifacts public
Requirements	Analysts translate needs	Users in the team	Developers = users
Staff Assignment	1 fixed project	1 fixed project	Multi-project, various levels
Code Review	Rare	Pair programming	Required, universal
Release Timelines	Few large releases	Small releases	Nightly, dev, stable
Organization	Top-down management	Self-organized	Each contributor decides
Testing	Separate QA	Integrated TDD	Everyone tests
Work Distribution	Assigned parts	Everyone modifies everything	Everyone modifies, limited commits

Care and Feeding of FOSS (Craig James)

FOSS Software Lifecycle (Free and Open Source Software)

1. **Invention**: initial idea, explosion of interest and new projects.
2. **Expansion and Innovation**: corporate competition, too many features, not suitable for open source.
3. **Consolidation**: the market shrinks, a few companies dominate.
4. **Maturity**: clear specifications, excellent moment for the birth of open projects (e.g., Linux).
5. **Transition to FOSS Domination**: companies begin using or integrating FOSS.
6. **FOSS Domination**: the open product begins eroding closed competitors.
7. **The FOSS Era**: complete dominance of the open market (utopian, never fully realized).

Examples

- **Linux**: born from Linus Torvalds after Minix, exploiting mature specifications.
- **IBM**: adopts Linux, abandoning its own OS.
- **Solaris**: made open-source but quickly forgotten.

Conclusion

- Open source excels in **continuous innovation and adaptation**.
- Large companies are slowed down by their traditional economic model.

The Emerging Economic Paradigm of Open Source (Bruce Perens)

Why Companies Adopt Open Source

- Often software is **an enabling technology**, not the product itself.
- Software development = necessary cost, not a direct source of revenue.

Competitive Advantage

- Proprietary software makes sense only if:
  1. The customer notices its effect, if that feature truly distinguishes us from others and is therefore best kept private.
  2. Competitors do not have access to the same software.
- If either answer is "no", open source is preferable:
  - It spreads costs.
  - It generates value through free contributions.

Software Development Models

- **Retail**: product sold to anyone.
- **In-House & Contract**: software commissioned for a specific client (e.g., Amazon corporate).
- **Consortium & Non-Open Collaboration**: competing companies collaborate (rare).
- **Open Source**: public and free code.

Comparison Criteria Efficiency, failure rate, distribution costs, plagiarism risk, protection of competitive advantage, minimum market size.

An empirical study of open-source and closed-source software products (Paulson)

Objective Empirically verify common assertions regarding open-source software.

Main Results

- Open-source projects show a **higher frequency of modifications** to functions.
- From this, it is not possible to deduce whether:
  - errors are found earlier (many modifications imply the error was seen earlier compared to a scenario where modifications are rarer? Not necessarily);
  - bugs are actually resolved faster.
- Some beliefs about open source turn out to be **only partially true**.

Challenges of the Open-Source Model

1. Software Integration

- **Historical problem**, amplified in FOSS.
- Previous models:
  - **Waterfall**: integration as a separate, rigid phase.
  - **Stabilize & Synchronize (Microsoft)**: nightly build; if it fails, it is fixed; if it succeeds, a new "best version" is established.
  - **XP**: continuous integration, only one developer at a time.
- **Open Source**: anarchic and continuous integration, without coordination.
- Risk: sudden large changes in the codebase.

2. Team Fragmentation

- Difficulties in communication, coordination, cohesion, and recruitment.
- Fundamental tools:
  - Internet and forums for communication.
  - Wikis (in the past) or **Pages** for documentation (Markdown, versioning with Git).
  - **Build automation** (e.g., Makefile) for reproducible environments.
  - **Bug tracking systems** accessible to all to prevent duplicates and manage reports.
- Necessary to educate users on correct bug reporting.

4 Software Configuration Management (SCM)

Software Configuration Management solutions arise to solve highly common problems in software development, such as:

- Publishing a *hotfix* (e.g., an urgent bug fix) on a version prior to the one currently under development;
- Managing concurrent access and conflicts among developers;
- Establishing responsibility for each line of code.

SCM is the set of practices that make the development process systematic, tracking every change so that the product is always in a well-defined state (**configuration**) and prior versions can be easily retrieved.

4.1 History

- **1950s**: originates in the aerospace sector.
- **1970s–1980s**: application to software engineering.
- **Objective**: maintain controlled revisions of *artifacts* and generate a product from defined configurations.

4.2 Artifacts and Configurations

The objects whose evolution is monitored are called **configuration items** or **artifacts** (generally files). Changing a file name is equivalent to deleting it and creating a new one.

Evolution of Tools

- **1980s**: local tools (`SCCS`, `RCS`);
- **1990s**: centralized client–server tools (`CVS`, `Subversion`);
- **2000s**: distributed peer-to-peer tools (`Git`, `Mercurial`, `Bazaar`).

4.3 Centralized vs Decentralized

centralized ⇒ working online on a server (one at a time, network bottleneck)
decentralized ⇒ everyone can download the repo offline

The **open source** world prefers decentralized approaches because:

- They allow working offline;
- They are faster (no network bottleneck);
- They support different working models:
  - Semi-centralized: a "reference" repository;
  - Peer-to-peer: direct collaboration;
  - Hierarchical multi-level (e.g., Linux kernel).

There is no automatic synchronization: merging between repositories is always explicit. In Git, custom merge algorithms can also be used.

4.4 Basic Mechanism

SCMs do not depend on the programming language: they work at the file level, treating each line as text (exception: **Monticello** for Smalltalk (development environment)).

Main Operations

- **Check-out**: declares the intention to work on a specific revision (like registering in a library to borrow a book);
- **Check-in / Commit**: registers a new revision (returning the book).

Everything occurs through data exchange between:

- **Repository**: contains versions, tags, branches, etc.;
- **Workspace**: local working environment.

4.5 Repository

The repository preserves differences between versions (*deltas*), but in Git, **symbolic links** are used for speed. Every version is accessible instantaneously.

Centralized vs Distributed Repository

- **Centralized**: a main server.
- **Distributed**: each developer has a complete copy, identified via hashes.

4.6 What to Track

The following are tracked:

- Source code;
- Configuration files.

It is not always beneficial to track:

- Compilers, external libraries (they already have their own versioning system);
- Generated binary files (very large and recomputed from source by the compiler).

This reduces perfect replicability but increases efficiency. Sometimes generated files are published as **packages** (e.g., on GitHub/GitLab).

4.7 Concurrent Access

Two main models:

- **Pessimistic** (e.g., RCS): only one user can write (lock);
- **Optimistic** (e.g., CVS): parallel work is allowed, and conflicts are resolved via merging.

In Git, the use of **branches** is encouraged and made efficient (e.g., GitFlow policies).

4.8 Distributed SCM — Advantages and Limitations

Advantages

- Offline work;
- Higher speed;
- Support for flexible collaboration models (peer, centralized, hierarchical).

Limitations

- Each peer has its own repository;
- No automatic synchronization → manual merges required.

4.9 Git

4-Level Structure

1. **Working Directory (WD)** — working folder on the filesystem (tracked and untracked files);
2. **Index / Staging Area** — temporary area for tracked files;
3. **Local Repository** — set of commits and their relative history;
4. **Remote Repository** — remote repository, can feature multiple branches or projects.

Transition between levels is always explicit (never automatic).

Commit pointers

- Each **branch** points to its last commit;
- **HEAD** points to the current commit;
- **Detached HEAD**: when visiting a previous commit without creating a branch.

4.9.1 `git commit - record changes to the repository`

- Saves content from the index to the local branch;
- After the commit, `HEAD` and the branch pointer update to the new commit;
- The `-amend` option allows replacing the last commit.

4.9.2 `git switch - switch branches`

- Simplified variant of `git checkout`;
- Changes branch by updating `HEAD`, `index`, and WD files.

4.9.3 `git merge - join development histories`

- Joins two (or more) branches:
  - If non-divergent → **fast-forward merge**;
  - Otherwise → **merge commit** or conflict.
- Conflicts reported explicitly:

```
<<<<<<< yours:sample.txt
Conflict resolution is hard;
let's go shopping.
=======
git makes conflict resolution easy.
>>>>>>> theirs:sample.txt
```

- After resolution → final merge commit.

3-way merge Git utilizes the **3-way merge**:

- Considers the two HEAD commits and the first common ancestor commit;
- Compares file differences;
- If the same part is modified in both → manual conflict.

4.9.4 `git reset - reset current HEAD`

- Moves `HEAD` (and optionally the WD) back to a previous commit;
- The `-soft` option (Index and WD remain unchanged);
- The `-mixed` option (default option, WD remains unchanged);
- The `-hard` option also restores local files.

—

*→ Consult the **Git Cheatsheet** for an operational summary of core commands.*

5 Git Workflow and Tools

5.1 Branches in Git

- Allow separate versions of code.
- Complete freedom regarding creation and naming.
- Necessary to establish policies for large teams.

5.2 GitFlow

- Branch organization and guided merging.
- Shell commands integrated into scripts perceived by Git.
- Main branches:
  - **master**: stable versions ready for production (each commit is a new version, e.g., end of iteration).
  - **develop**: integration of features (each commit is a new feature).
- Temporary branches:
  - **feature**: new functionalities or bug fixes; stem from develop (one branch per feature).
  - **release**: preparation for release, targeted bug fixes; stem from develop (a branch used to fix general bugs to arrive at a stable version for master).
  - **hotfix**: urgent corrections; stem from master (e.g., bug fixes after release).

5.2.1 Feature Branches

- Isolate work on functionalities or bugs.
- Merge into develop with: `git merge -no-ff feature/myFeature` (joins the feature/myFeature branch to the current branch).
- –no-ff avoids fast-forward to track feature start/end (if omitted and no conflicts exist, even if we add new files, the result would just move the pointer without creating an actual commit, which is why it is generally used).

- Squash optional (merging all commits of the feature branch into a single final commit), but recommended *no* to preserve commit history.

5.2.2 Release Branches

- Crystallizes functionalities present in develop.
- Merge into master (release) and develop (update).
- Bug fixes possible with cherry-pick (extracting a single commit from develop rather than all modifications, e.g., fixing a bug in one commit while previous commits introduced functionalities that might not be tested yet).
- Tagging with `git tag -a vX.Y.Z` (to mark versions).

5.2.3 Hotfix Branches

- Urgent corrections on master and develop.
- Merge into master and develop; tags and branches are deleted at the end.

5.3 Limitations of Git and GitFlow

- Lack of granular permissions.
- Absence of code review.
- No integrated communication system.

5.4 git request-pull

- Generates a proposal of changes via email.
- Replaced today by web pull requests.
- Syntax: `git request-pull [-p] <start> <URL> [<end>]`

5.5 Centralized Hosting

- Central repository for integration and control.
- GitHub/GitLab offer forks, pull requests, CI/CD, and permissions.
- **Fork**: copy of the original repository, independent but linked to the parent.

5.6 Pull Requests / Reviews

- Facilitate interaction and review.
- Can generate conflicts if code evolves in parallel.
- CI (Continuous Integration) automates testing since it automatically verifies that new code does not break already implemented functions, immediately signaling possible errors (tests are not generated automatically), but manual review remains recommended.

5.7 Gerrit

- Peer-to-peer review system (Google, Android).
- Modification accepted only as a single commit (changeset).
- Key figures:
  - **Verifier**: automatic patch verification.
  - **Approver**: approves validity, architecture, security, best practices.
- Architecture:
  - Authoritative repository: anyone can only read (hence copy the repo); only after reviewers have approved changes (at least 3 reviewers) are the modifications automatically added to the repo through pending changes.
  - Pending Changes: all developers can submit pull requests; reviewers examine and accept or decline them.

5.8 Open Source Tools

- **Build automation**: make, Ant, Gradle.
- **Bug tracking**: git-bug, BugZilla, Scarab, GNATS, BugManager, Mantis.

5.8.1 Build Automation

- Automates compilation and deployment.
- **make**: shell scripts, incremental compilation, dependency management.
- **Makefile**: defines targets, dependencies, and commands.
- Automatic generation: automake, autoconf, imake.
- **Ant**: XML, Java, portable across all platforms (limiting a Makefile since it depends on OS commands, built for Linux), integrated deployment, tasks with dependencies.
- **Gradle**: Groovy/Kotlin, higher level compared to Ant, multi-project, Maven dependency management, plugins for tasks.

5.8.2 Bug Tracking

- Helps programmers know how to handle a bug and follows a precise modus operandi; an issue is opened, explaining exactly what the bug causes.
- A bug will then be examined and can be closed for the following reasons:
  1. duplicate: error already reported.
  2. wontfix: error that there is no intention to resolve (intended design characteristics).
  3. can't reproduce: poorly explained or non-reproducible bug.
  4. fixed: fixed but not yet integrated.
  5. fix verified: fixed, tested, and integrated.
- bug considered eliminated only when the fix enters master.

5.9 Unified Process (UP / RUP)

- OO (Object Oriented) metamodel for software development.
- Sequential with 4 phases:
  1. Inception: study phase, estimation, and goal stabilization.
  2. Elaboration: establishing the core architecture.
  3. Construction: developing, testing, and integrating.
  4. Transition: deployment phase to the client and correction of residual bugs.
- Iterative: development is not done all at once but little by little.
- Incremental: partial release and new iteration.
- Overlapping workflows (4 separate phases where classical phases like requirements, design, implementation are executed together but in varying measures, e.g., in the first phase, a lot of requirements and design are done), milestones marking the end of each phase.
- Configurable and adaptable, can become agile.

5.10 Refactoring and Design Principles

Refactoring Rules:

- Do not modify functionality ⇒ invisible to the client.
- Do not insert new tests after the green phase.
- If it takes too long ⇒ rollback to the last green version and plan multiple steps.
- *"Do it twice" rule*: the second attempt is faster and better.

Why should I do it?:

- Complex/unreadable design.
- Prepare design for new features that are difficult to integrate (*design for change*).
- Presence of *technical debt*: shortcuts that slow down evolution.

Design Knowledge (how to keep track of how code is structured):

- **Memory:** unreliable over time.
- **Design documents (diagrams):** become desynchronized if not updated.
- **Discussion platforms:** scattered info, difficult to update.
- **UML and generative programming:** ineffective in practice.
- **Code:** most reliable source, but difficult to understand the motivations.

SOLUTION: Combine multiple techniques; document the reasons for choices, not the choices themselves.

Design Sharing:

- **Methods:** (e.g., Agile, OO).
- **Design patterns:** common language, reusability, known solutions.
- **Principles:** e.g., SOLID.

6 Preliminary Knowledge

6.0.1 Object Orientation Language

Three fundamental properties:

- **Inheritance:** inherits properties and behaviors.
- **Polymorphism:** classes assume different forms based on interfaces.
- **Dynamic binding:** occurs at runtime in Java; in C++ it requires `virtual` (operation to realize polymorphism).

6.0.2 SOLID Principles

- **S - Single Responsibility:** one class, one purpose only.
- **O - Open/Closed:** extendable without internal modifications.
- **L - Liskov Substitution:** subclasses respect the contracts of the parent class.
- **I - Interface Segregation:** many small interfaces, not one large one.
- **D - Dependency Inversion:** depend on abstractions, not on concretions.

6.0.3 Reference Escaping (error to avoid)

Violation of encapsulation:

- Getter returning a private reference.
- Setter/constructor assigning external references (if the reference to the passed object is stored, I can modify that object externally; if I make a copy instead, I cannot).

```
1              public void setDataInizio(Date nuovaData)
                   {
2                  if (nuovaData == null) {
3                      throw new
                           IllegalArgumentException("La
                           data non puo essere null");
4                  }
5                  // Defensive copy to prevent
                      reference escaping
```

```
6                   this.dataInizio = new Date(nuovaData.
                        getTime());
7               }
```

6.0.4 Encapsulation and Information Hiding

Parnas's Law: only what is hidden can be modified freely. Clearly separate what is public from what is private.

6.0.5 Legacy and Deprecated

- **Legacy:** supported but replaced by a more recent version.
- **Deprecated:** working but unsupported; to be replaced with new APIs.

6.0.6 Immutability

Immutable class ⇒ state cannot be modified after creation.

- No mutator methods.
- Private and/or final attributes.
- Getters that do not return references.
- No reference escaping.

6.0.7 Code Smells

Signs of poor design (technical debt):

- Duplicated code.
- Long methods / too many indentation levels.
- Too many attributes.
- Sequences of if/switch ⇒ prefer polymorphism.
- Long parameter lists.
- Magic numbers ⇒ use named constants.
- Explanatory comments ⇒ unclear code.
- Obscure or inconsistent names.

- Dead or untested code.
- Getters/Setters ⇒ loss of encapsulation.

6.0.8 Tell-Don't-Ask

Do not ask for data, but tell what to do with it. The object must manage its own information and expose only operations, not raw data.

6.0.9 Interface Segregation (practical approaches)

- **Up front:** writing the interface directly.
- **Down front:** writing the code and then extracting the interface (more suited to TDD).

6.1 Example: `Card / Deck` hierarchy (up front)

Concepts covered: Interface segregation, static/dynamic binding, loose coupling, multiple interfaces, contract-based design vs defensive programming, abstract classes.

Interface Segregation in Action

- Method:

```
1  public static List<Card> drawCards(
       CardSource deck, int n) {
2      List<Card> result = new ArrayList<>()
           ;
3      for (int i = 0; i < n && !deck.
           isEmpty(); i++)
4          result.add(deck.draw());
5      return result;
6  }
```

- `Deck` implements only the necessary capabilities: `isEmpty()` and `draw()`.
- The interface `CardSource` abstracts these capabilities, allowing generic reuse.

Static and Dynamic Binding

- In Java ⇒ statically declared type: `Deck implements CardSource`.
- In Go ⇒ more dynamic: matching methods are sufficient (no need to explicitly declare an object as a subtype of another, it just needs to have the same method name).
- Duck typing risk: vague names ⇒ ambiguity (e.g., `play()` in different contexts).

Loose Coupling

- Allows assigning different types to general variables as long as they are subtypes:

```
1  Deck deck = new Deck();
2  CardSource source = deck;
3  List<Card> cards = drawCards(deck, 5);
```

- Promotes flexibility and extensibility.

Multiple Interfaces

- `Deck` can implement multiple interfaces (`CardSource`, `Shuffable`, `Iterable<Card>`).
- Each method uses only the capabilities that are truly required.

Contract-based Design vs Defensive Programming

- **Contract-based:** contract between caller and callee.
  - The caller must respect the preconditions (`@pre !isEmpty()`).
  - Contract violation = logical error.
- **Assertions:** useful during development, removed in production.
- **Defensive programming:** validation is the responsibility of the callee, not the caller.

Abstract Classes

- Can partially implement an interface and delegate the rest to subclasses.
- Cannot be instantiated; improve generality but with a slight loss in performance.
- Useful also for complete classes that conceptually shouldn't be instantiated (e.g., utilities).

Example: Java Utility Classes

- `Collections.shuffle(List<?> list)`: ignores type, only shuffles.
- `Collections.sort(List<T> list)` requires:

```
public static <T extends Comparable<? super T>> void sort(List<T> list)
```

- `Comparable<T>` specifies that an object is comparable.
- Comparable is an example of *interface segregation*: an interface for a single capability.

Final Digression

- The `Collections` class was a way to add methods to interfaces.
- Today, `default methods` can be used directly inside interfaces (interfaces generally do not contain function bodies, but default allows doing so).

7 Design Patterns

7.1 General Definition

- Conceptual, reusable solutions to recurring design problems.
- Not code itself but **design architectures** recognized as effective.
- Each pattern must have an evocative name to aid memorization.
- **Idioms** are concrete implementations of a pattern within a specific language.
- There are also **anti-patterns**, which are solutions that appear good but have been proven incorrect.

7.2 Pattern Categories (GoF)

- **Creational**: managing object creation.
- **Structural**: composition of classes and objects.
- **Behavioral**: interaction and communication between objects.

7.3 Meta-patterns

- Patterns for **building other patterns**.
- Two fundamental elements:
  - **Hook Method**: abstract method representing the hot, modifiable spot.
  - **Template Method**: general structure using hooks to define the variable part.

  An example could be a sorting function, which contains a general skeleton of the algorithm (template) within which the method of comparing elements is not specified. Customizing this comparison implements the hooks, which when inserted into the template make the algorithm functional.

- Hook–Template Connection:
  - **Unification**: hook and template inside the same class (e.g., `Iterable` in Java, which requires implementing next and hasNext methods; furthermore, there is a default method called forEach that allows calling next as long as hasNext returns true. All these methods reside in the same interface).
  - **Connection**: hook and template in different classes connected by aggregation (template class containing the hook class).
  - **Recursive Connection**: hook and template in classes connected by generalization (e.g., Decorator, Composite).

7.4 Singleton

- Guarantees that a class has only one instance.
- Classical implementation:
  - Private/protected constructor.
  - Static method `getInstance()` that creates and returns the single instance, checking if it has already been instantiated.

```
1  public class Singleton {
2      /* private or otherwise non-public
           constructor */
3      protected Singleton() { ... }
4
5      /* store the instance to use later */
6      private static Singleton instance =
           null;
7
8      /* static method */
9      public static Singleton getInstance()
           {
10         if (instance == null) {
11             instance = new Singleton();
12         }
13         return instance;
14     }
15
16     public void metodoIstanza() { ... }
17 }
```

- Problem: concurrency (if the method is accessed simultaneously, duplicates can occur) → use of the **Double Check Pattern** with a `synchronized` block; only a portion is synchronized after checking whether it has already been instantiated, preventing severe performance drops.

```
public class Singleton {
    private static Singleton instance =
       null;

    protected Singleton() {}

    public static Singleton getInstance()
        {
        if (instance == null) {
            synchronized(Singleton.class)
                {
                if (instance == null)
                instance = new Singleton
                   ();
            }
        }
        return instance;
    }
    public void sampleOp() {...}
}
```

- **Java Idiom**: using an `enum` with a single value (`INSTANCE`), which is thread-safe.

```
public enum MySingleton {
    INSTANCE;

    public void metodoIstanza() { ... }
}

MySingleton.INSTANCE.sampleOp();
```

- Disadvantages: can violate SOLID principles and act as a hidden global variable.

7.5 Iterator

- Allows traversing elements of a collection without exposing its internal representation.
- Interfaces:
  - `Iterator<E>`: defines `hasNext()`, `next()`, `remove()`.
  - `Iterable<E>`: defines `iterator()`, enabling the `for-each` construct (in practice, the compiler translates for-each into iterator syntax if iterable is implemented, but this can only be done on a single type).
- Problem: `remove()` violates SRP → it would be better to separate it.
- A class cannot be iterable over multiple different types (type erasure).

7.6 Chain of Responsibility

- Handles sequential requests within a **chain of handlers**.
- Each handler decides either to process the request or pass it to the next one.
- Avoids long `if/else` blocks and simplifies extensibility.
- Implementation:
  - Interface `Gestore` with method `evaluate()`.
  - Each handler contains a reference to the next (`next`).
  - The last one can use a **NullObject** as a terminator.

```
1   public interface Gestore {
2
3       /* Return type depends on application scope
           */
4       public ??? evaluate();
5
6   }
7
8   public class Client {
9
10      private Gestore evaluator =
```

```
11              new GestoreConcreto1(
12              new GestoreConcreto2(
13              new GestoreConcreto3(null)));
14
15              public void richiesta() {
16                  evaluator.evaluate();
17              }
18
19          }
20
21          public class GestoreConcreto {
22
23              private evaluator next;
24
25              public GestoreConcreto(evaluator next) {
26                  this.next = next; // next could also be
                        null
27              }
28
29              public GestoreConcreto() {
30                  this.next = null;
31              }
32
33
34              public ??? evaluate() {
35                  if thisIsTheCase() return "Questo
                        evaluatore gestisce il caso";
36                  else
37                  if next != null return next.evaluate();
                        // try with next
38                  else return "Non sono in grado di gestire
                         questo caso";
39                  // reached the end of the chain and
                        couldn't handle it
40              }
41
42          }
```

7.7 Flyweight

(similar to singleton, the difference is that flyweight has a shared immutable internal state across other classes to save memory)

- Manages shared immutable objects to reduce memory usage.
- Private constructor, creation of all instances in advance, stored in a static structure.
- Clients request instances via a `get(...)` method that always returns the same one.
- Example: `Card` class containing all combinations of suits and ranks.
- Advantage: memory efficiency; disadvantage: heavy initial setup.

```
1  public class Card {
2      private static final Card[][] CARDS = new
           Card[Suit.values().length][Rank.values().
           length];
3      // the static { ... } block performs the
           exact job of a constructor, but at the
           class level
4      static {
5          for (Suit suit : Suit.values()) {
6              for (Rank rank : Rank.values()) {
7                  CARDS[suit.ordinal()][rank.
                       ordinal()] = new Card(rank,
                       suit);
8              }
9          }
10     }
11
12     public static Card get(Rank pRank, Suit pSuit
           ) {
13         return CARDS[pSuit.ordinal()][pRank.
               ordinal()];
14     }
15 }
```

7.8 Null Object

- Avoids explicit use of `null` and `NullPointerExceptions`.
- Approaches:
  - Conditional checks or assertions.
  - Annotations (`@NotNull`).
  - Use of **Optional<T>** to represent absent values.

```
public interface CardSource {
    Card draw();
    boolean isEmpty();
    // instance that defines itself as
       null with the interface logic
    public static CardSource NULL = new
       CardSource() {
        public boolean isEmpty() {
            return true;
        }
        public Card draw() {
            assert !isEmpty();
            return null;
        }
    }
}
```

- **NullObject** Pattern: defines a static instance `NULL` with neutral behavior (in practice, when implementing this interface, writing cardSource.NULL will yield the same object but null).
- Preserves type identity and prevents runtime exceptions.

7.9 Strategy (Delegation)

- Defines a **family of interchangeable algorithms** at runtime.
- Replaces inheritance with composition.
- Each behavior is encapsulated in a `Strategy` object (if a duck has both flying and quacking behaviors, it is possible that some ducks cannot fly while others can; to avoid implementing fly in each duck, common methods and optional ones are separated into different interfaces, establishing implementable behavior policies).

- The client class delegates the action to a strategy object (e.g., `FlyBehaviour` inside `Duck`).
- Advantages:
  - Higher flexibility and maintainability.
  - Adherence to Open/Closed and Liskov principles.

7.10 Observer

- Keeps multiple objects synchronized with a common state.
- Avoids the anti-pattern of **pairwise dependencies** (3 classes sharing a state where A depends on B, B on C, and C on A; changing one requires changing the other two) because the state is distributed.
- Structure:
  - `Subject` (observed) maintains the state and the list of observers.
  - `Observer` registers and receives notifications upon changes.
- **Event-driven** architecture: the Subject notifies, it is not polled.
- two ways to do it:
  - `Push`: sending the entire state directly without passing the observer instance (PROBLEM: state might be heavy, making passing everything useless).

```
1                   // Observable
2                   @Override
3                   public void notifyObservers() {
4
5                       for (Observer observer :
                           observers) {
6                           observer.update(null,
                               state);
7                       }
8
9                   }
10
11                  // Observer
12                  @Override
```

```
13                  public void update(Observable
                        model, Object state) {
14
15                      if (state instanceof Integer
                            intValue) {
16                          doSomethingOn(intValue);
17                      }
18
19                  }
```

– `Pull`: passing only the observable instance and no state, letting the observer fetch only what interests it via getters of various states (PROBLEM: casting is required to verify that the passed observable matches the requested one (`model instanceof ConcreteObservable cMod`

```
1                   // Observable
2                   @Override
3                   public void notifyObservers() {
4
5                       for (Observer observer :
                            observers) {
6                           observer.update(this,
                                null);
7                       }
8
9                   }
10
11                  // Observer
12                  @Override
13                  public void update(Observable
                        model, Object state) {
14
15                      if (model instanceof
                            ConcreteObservable cModel)
                             {
16                          doSomethingOn(cModel.
                                getState());
17                      }
18
19                  }
```

- `hybrid approach`: often a hybrid approach between pull and push is used, passing the most relevant state while allowing extraction of

specific state details.

- `resolving casting`: often an attribute representing the observable is placed in the observer to avoid casting, using only the check:

```
public class ConcreteObserver {
    private Observable mySubject;

    @Override
    public void update(Observable subject
       , Object stato) {
        if (subject == mySubject) {
            ...
        }
    }
}
```

- In Java: `Observer`, `Observable` interfaces (deprecated, not thread-safe).

7.11 Adapter

Purpose: make incompatible interfaces or components compatible when not originally designed to work together (e.g., COTS libraries or legacy code). Allows incremental and controlled integration of heterogeneous components.

Types:

- **Class Adapter:** adapts a *class*.
- **Object Adapter:** adapts an *object*.

7.11.1 Class Adapter

Idea: the `Adapter` class **extends** `Adaptee` (old class to be adapted) and **implements** `Target` (interface with new methods that will adapt the old class). The client uses the `Adapter` via the `Target` interface, but calls are forwarded to the methods of the adapted class.

```
public class Adapter extends Adaptee implements
   Target {
    @Override
    public void request() {
        this.oldRequest();
    }
```

```
6         }
```

Pros:

- Direct access to protected methods/attributes of the Adaptee.
- Reuse of existing code.
- Possibility to use the instance with both interfaces.

Cons:

- Problems with multiple inheritance (e.g., Java).
- Difficult to adapt an interface (no code inheritance since interfaces contain no implementation).
- Possible name clashes between identical method names with different behaviors.

7.11.2 Object Adapter

Idea: the `Adapter` class **contains** (composition) an instance of `Adaptee` and **delegates** calls to it.

```
1         public class Adapter implements Target {
2             private final Adaptee adaptee;
3             public Adapter(Adaptee adaptee) { this.
                  adaptee = adaptee; }
4             @Override
5             public void request() { adaptee.oldRequest();
                  }
6         }
```

Pros:

- Supports adaptation of interfaces (does not use inheritance).
- No multiple inheritance issues.
- Can adapt entire class hierarchies.

Cons:

- Requires two instances (Adapter + Adaptee) → higher memory usage.
- No access to protected members of the Adaptee.
- Must reimplement all methods (explicit delegation).

7.11.3 Comparison: Class vs Object Adapter

Aspect	Class Adapter	Object Adapter
Access to Adaptee	Direct (protected and public)	Only via public interface
Code Reuse	High (no reimplementation)	Low (explicit delegation)
Memory Usage	1 instance	2 instances
Interface Usage	Old and new	New only
Multiple Inheritance	Problematic	None
Interface Adaptation	Not indicated	Excellent, even for hierarchies

Facade

- **Problem:** a complex system can expose many specific and detailed interfaces, forcing the client to interact with multiple subsystems even for simple operations.
- **Soluzione:** provide a **unified and simplified interface** (the Facade) that hides internal complexity by combining various functionalities.
- **Purpose:** simplify system usage and make access to frequent operations immediate, while preserving the possibility to use more granular interfaces.
- **Examples:**
  - **Remote control**: controls TV with simple commands, but allows access to advanced functions.
  - **GitFlow**: commands like |init|, |feature start/finish| or |release| execute multiple operations (branch, merge, etc.) behind a simple interface.

Composite

- **Problem:** modeling hierarchical structures (e.g., file system with Files and Directories) where the same operations (create, delete, calculate size) must work uniformly.
- **Solution:** introduce the **Composite** pattern, which handles single objects and composed objects uniformly via a **common interface** (`Component`).
- **Structure:**
  - `Component`: common interface for operations.

  - `Leaf`: single object.
  - `Composite`: contains 0..n `Components` (Leaf or Composite), implements common operations by delegating them to internal components.

- **Cardinality:**
  - `Composite` → `Component`: 0..n (can contain multiple components).
  - `Component` → `Composite`: 0..1 (each component can belong to only one Composite).

- **Advantages:** uniform management of objects and groups; simpler code for recursive operations.

- **Disadvantages:** impossibility to distinguish clearly between Leaf and Composite; no control over tree content or depth.

- **Solutions to distinguish types:**
  - Method returning the real type (`this` or `null`) → unclean.
  - Controlled casting with `instanceof` → acceptable but not ideal.
  - Moving all operations to the interface and throwing exceptions in Leafs → inelegant.

- **Example: Composite Deck**
  - `CardSourceComp` contains a list of `CardSources`.
  - `isEmpty()` returns true only if all sub-decks are empty.
  - The `draw()` method becomes more complex (requires a random choice of deck).

```
1  public interface CardSource {
2      Card draw();        // To draw a card
3      boolean isEmpty();  // To check if empty
4  }
```

```
1  public class CardSourceComp implements CardSource
       {
2      @NotNull private List<CardSource> sources;
3
4      public CardSourceComp(@NotNull List<
           CardSource> sources) {
```

```
5                   this.sources = sources;
6               }
7               @Override
8               public boolean isEmpty() {
9                   for(CardSource source : sources) {
10                      if(!source.isEmpty()) return false;
11                  }
12                  return true;
13              }
14              @Override
15              public Card draw() {
16                  for (CardSource source : sources) {
17                      if (!source.isEmpty()) {
18                          return source.draw();
19                      }
20                  }
21                  return CardSource.NULL.draw();
22              }
23          }
```

```
1           public class SingleDeck implements CardSource {
2               private final String name;
3               private int cardCount;
4
5               public SingleDeck(String name, int
                    initialCount) {
6                   this.name = name;
7                   this.cardCount = initialCount;
8               }
9
10              @Override
11              public Card draw() {
12                  if (cardCount > 0) {
13                      cardCount--;
14                      System.out.println(name + " ha
                            pescato una carta. Rimanenti: " +
                            cardCount);
15                      return new Card();
16                  }
17                  return CardSource.NULL.draw();
18              }
19
20              @Override
```

```
21              public boolean isEmpty() {
22                  return cardCount == 0;
23              }
24          }
```

Decorator

- **Problem:** dynamically adding functionalities to an object without an explosion of classes or violating the Open-Closed Principle.
- **Incorrect solutions:**
  - Static hierarchy of classes for each combination → **anti-pattern** (combinatorial explosion).

```
public interface Pizza {}
public class BaseNormale
   implements Pizza {
    public String toString() {
        return "Sono una pizza
           con: base normale";
    }
}
public class BaseIntegrale
   implements Pizza {
    public String toString() {
        return "Sono una pizza
           con: base integrale";
    }
}
public class BaseNormaleSalame
   extends BaseNormale {
    public String toString() {
        return "Sono una pizza
           con: base normale,
           salame";
    }
}
public class
   BaseNormaleSalamePeperoni
   extends BaseNormaleSalame {
    public String toString() {
        return "Sono una pizza
           con: base normale,
           salame, peperoni";
    }
}
```

  - `God Class` with boolean flags → violation of OCP, poor maintainability.

```
1  public class GodPizza {
2      boolean baseNormale = false;
3      boolean baseIntegrale = false
           ;
4      ...
5      boolean salame = false;
6      boolean pancetta = false;
7      boolean peperoni = false;
8      ...
9
10     public void setBaseNormale(
           boolean status) {
           baseNormale = status; }
11     public void setBaseIntegrale(
           boolean status) {
           baseIntegrale = status; }
12     ...
13
14     public void setSalame(boolean
           status) { salame = status
           ; }
15     public void setPancetta(
           boolean status) { pancetta
           = status; }
16     public void setPeperoni(
           boolean status) { peperoni
           = status; }
17     ...
18
19     public String toString() {
20         StringBuilder sb = new
               StringBuilder("Sono
               una pizza con: ");
21         if (baseNormale) sb.
               append("base normale,
               ");
22         if (baseIntegrale) sb.
               append("base integrale
               , ");
23         ...
24         if (salame) sb.append("
               salame, ");
```

```
25                          if (pancetta) sb.append("
                               pancetta, ");
26                          if (peperoni) sb.append("
                               peperoni, ");
27                          ...
28                          sb.removeCharAt(sb.length
                               () - 1);
29                          sb.removeCharAt(sb.length
                               () - 1);
30                          return sb.toString();
31                      }
32                  }
```

- **Solution:** introduce **Decorator Pattern**.
- **Structure:**
  - `Component`: common interface.
  - `ConcreteComponent`: base (e.g., `BaseNormale`).
  - `Decorator` (abstract): implements `Component`, contains reference to another `Component`.
  - `ConcreteDecorator`: adds functionality before/after calls to the internal component.

```
1               public interface Pizza { String toString
                    (); } // component
2
3               public class BaseNormale implements Pizza
                    { // concreteComponent
4                    public String toString() {
5                        return "Io sono una pizza con:
                             base normale";
6                    }
7               }
8
9               public class BaseIntegrale implements
                    Pizza { // concreteComponent
10                   public String toString() {
11                       return "Io sono una pizza con:
                             base integrale";
12                   }
13              }
```

```
14
15              public abstract class
                    IngredienteDecorator implements Pizza
                    { // decorator
16                   private Pizza base;
17
18                   public IngredienteDecorator(Pizza
                        base) { this.base = base; }
19
20                   public String toString() {
21                       return base.toString();
22                   }
23              }
24
25              public class IngredienteSalame extends
                    IngredienteDecorator { //
                    concreteDecorator
26                   public IngredienteSalame(Pizza base)
                        { super(base); }
27
28                   @Override
29                   public String toString() { return
                        super.toString() + ", salame"; }
30              }
31
32              public class IngredientePeperoni extends
                    IngredienteDecorator { //
                    concreteDecorator
33                   public IngredientePeperoni(Pizza base
                        ) { super(base); }
34
35                   @Override
36                   public String toString() { return
                        super.toString() + ", peperoni"; }
37              }
```

and then the client:

```
1               public class Client {
2                   public static void Main() {
3                       // I want a pizza with salami,
                            peppers, and whole wheat base
4                       Pizza salamePeperoni =
5                       new IngredientePeperoni(
```

```
                        new IngredienteSalame(
                        new BaseIntegrale()
                        )
                        );
                    }
                }
```

- **Differences with Composite:**
  - **Composite:** creates composed objects.
  - **Decorator:** adds functionality dynamically.
- **Characteristics:**
  - Each Decorator can decorate other Decorators → recursive structure.
  - Reference escaping intentional (the Decorator must reflect changes of the base object).
  - Ease of extension thanks to abstract class `Decorator` with `protected` method.
- **Combined use with Composite:** possible to decorate groups of objects or decorated groups (e.g., multiple effects in photo-editing).
- **Real-world example:** Java's `InputStream`.

State (finite state machines)

- **Purpose:** model objects whose behavior changes based on their current state.
- **Problem:** avoid using long `if`/`switch` blocks to manage states.
- **Structure:**
  - `State`: interface with methods representing actions.
  - `ConcreteState`: implements state-specific behavior.
  - `Context`: holds reference to the current state and delegates operations to it.
- **Transitions:**

  - **Managed by State:** more consistent with finite state automata, but introduces dependency between states.
  - **Managed by Context:** centralizes transitions, but reintroduces `if`/`switch`.

- **Preference:** let States handle them → atomic and clear state transitions.

- **Properties:**
  - States are reusable across different Contexts.
  - Each Context only stores the reference to the current state.

- **Differences with Strategy:**
  - `States` know about other states.
  - `Strategy` instances are independent.

```
// The State interface
public interface TrafficLightState {
    void handle(TrafficLightContext
        context);
}
```

```
// ConcreteState
public class RedState implements
    TrafficLightState {
    @Override
    public void handle(
        TrafficLightContext context) {
        System.out.println("Semaforo
            ROSSO: Fermati.");
        // Transition logic: next state
            is Green
        context.setState(new GreenState()
            );
    }
}

// ConcreteState
public class GreenState implements
    TrafficLightState {
    @Override
```

```
14                  public void handle(
                        TrafficLightContext context) {
15                      System.out.println("Semaforo
                            VERDE: Vai.");
16                      // Transition logic: next state
                            is Yellow
17                      context.setState(new YellowState
                            ());
18                  }
19              }
20
21              // ConcreteState
22              public class YellowState implements
                    TrafficLightState {
23                  @Override
24                  public void handle(
                        TrafficLightContext context) {
25                      System.out.println("Semaforo
                            GIALLO: Attenzione, preparati
                            a fermarti.");
26                      // Transition logic: next state
                            is Red
27                      context.setState(new RedState());
28                  }
29              }
```

```
1               // The Context class (The Traffic Light)
2               public class TrafficLightContext {
3                   private TrafficLightState state;
4                   // Initialization with the initial
                        state (e.g., Red)
5                   public TrafficLightContext() {
6                       this.state = new RedState();
7                   }
8
9                   // Method to change state (used by
                        State classes)
10                  public void setState(
                        TrafficLightState newState) {
11                      this.state = newState;
12                  }
13
14                  // Operation method delegates to
```

```
                    current State object
15              public void change() {
16                  // Call to handle() triggers
                        state-specific behavior
17                  state.handle(this);
18              }
19          }
```

Factory Method

- **Problem:** create objects without knowing their concrete class, but only the interface (I only need to know the vehicle factory, so if the vehicle changes, the client doesn't need to know).
- **Solution:** introduce a **virtual factory method** that delegates creation to subclasses.
- **Structure:**
  - `Creator` (abstract): defines `factoryMethod()` returning a `Product`.
  - `ConcreteCreator`: implements the factory method, instantiating a `ConcreteProduct`.
  - `Product`: common interface.
  - `ConcreteProduct`: real implementation.

```
1           // abstract creator
2           public abstract class VeicoloFactory {
3               public abstract Veicolo creaVeicolo()
                    ;
4
5               public String pianificaLogistica() {
6                   Veicolo veicolo = creaVeicolo();
                        // Call to factory Method
7                   return "Logistica pianificata.
                        Inizio: " + veicolo.guida();
8               }
9           }
```

```
1           // concrete creator
2           public class AutoFactory extends
                VeicoloFactory {
```

```
3                   @Override
4                   public Veicolo creaVeicolo() {
5                       return new Auto(); // Instantiates specific product: Auto
6                   }
7               }
8               // concrete creator
9               public class CamionFactory extends VeicoloFactory {
10                  @Override
11                  public Veicolo creaVeicolo() {
12                      return new Camion(); // Instantiates specific product: Camion
13                  }
14              }
```

```
1               // product
2               public interface Veicolo {
3                   String guida();
4               }
```

```
1               // ConcreteProduct 1
2               public class Auto implements Veicolo {
3                   @Override
4                   public String guida() {
5                       return "Guidando un'Auto: trasporto 4 persone.";
6                   }
7               }
8
9               // ConcreteProduct 2
10              public class Camion implements Veicolo {
11                  @Override
12                  public String guida() {
13                      return "Guidando un Camion: trasporto merce pesante.";
14                  }
15          }
```

CLIENT:

```
public class LogisticaClient {
    public static void main(String[] args) {

        // 1. Using the Auto Factory
        VeicoloFactory autoCreator = new AutoFactory();
        String result1 = autoCreator.pianificaLogistica();

        System.out.println("Scenario 1: " + result1);
        // Output: Scenario 1: Logistica pianificata. Inizio: Guidando un'Auto: trasporto 4 persone.


        // 2. Using the Camion Factory
        VeicoloFactory camionCreator = new CamionFactory();
        String result2 = camionCreator.pianificaLogistica();

        System.out.println("Scenario 2: " + result2);
        // Output: Scenario 2: Logistica pianificata. Inizio: Guidando un Camion: trasporto merce pesante.
    }
}
```

- **Characteristics:**
  - Allows polymorphism and late binding during creation (vehicle type established at runtime).
  - Factory methods cannot be static (if static, subclasses couldn't override them).
  - Also known as **virtual constructors**.
- **Example:** applications like Word/Excel:
  - `Application` (Creator) handles common logic.

- `MyApplication` (ConcreteCreator) implements `createDocument()`.

Abstract Factory

- **Purpose:** create **families of compatible objects** without specifying their concrete classes (positioned one step above factory methods, used when there are different types but similar structures, e.g., Tesla vs Ford car).

```
// Abstract Factory
interface GUIFactory {
    auto createAuto();
}
class TeslaFactory implements GUIFactory
   {
    public Auto createAuto() {
        return new Tesla();
    }
}
```

- **Problem:** need to create related objects (e.g., UI elements with the same style) ensuring coherence between them.
- **Solution:** introduce **AbstractFactory** with a factory method for each abstract product type.
- **Structure:**
  - `AbstractFactory`: interface with methods to create each `AbstractProduct`.
  - `ConcreteFactory`: implements creation of mutually compatible `ConcreteProducts`.
  - `Product`: common interface for created objects.
  - `ConcreteProduct`: concrete instances consistent with the factory.
- **Example: Cross-platform GUI**
  - `GUIFactory` with `createButton()`, `createCheckbox()`.
  - `MacFactory` → `MacButton`, `MacCheckbox`.
  - `WinFactory` → `WinButton`, `WinCheckbox`.
  - The client only knows the factory suited for the operating system and uses it to create all components.

7.12 Model View Controller (MVC)

Problem: the same information must be displayed with different views (e.g., color: RGB, hexadecimal, slider). Need to maintain **consistency across views** when the user modifies data.

- **Naive solution:** views updating each other → tight coupling, difficult maintenance (if I have a road length, I would have a view in meters and one in miles).
- **MVC solution:** separate *data*, *presentation*, and *interaction logic*.

Main Components:

- **Model:** contains the shared state and manages data. Single source of truth.
- **View:** represents the user interface. Displays data and receives input.
- **Controller:** manages application logic and interaction. Receives input from the View and updates the Model.

Typical Interaction Cycle:

1. The user interacts with a View → event sent to the Controller.
2. The Controller processes the event and (if necessary) modifies the Model.
3. The Controller can immediately update the View (e.g., error messages).
4. The Model changes state and **notifies all registered Views**.
5. Views update displayed data by fetching it from the Model (|pull policy|).

```
        // Observer Contract (The View will implement it)
        public interface Observer {
            void update();
        }

        // Subject Contract (The Model will implement it)
        public interface Subject {
            void registerObserver(Observer o);
            void removeObserver(Observer o);
            void notifyObservers();
        }
```

```
        // model
        public class UserModel implements Subject {
            private String nome;
            private String email;
            // List of all interested Views (the
               Observers)
            private List<Observer> observers = new
               ArrayList<>();

            public void loadUser(int userId) {
                this.nome = "Mario Rossi";
                this.email = "mario.rossi@example.com";
                // Upon loading, notify for initial
                   display
                notifyObservers();
            }

            // Key method of the Subject
            @Override
            public void registerObserver(Observer o) {
                observers.add(o);
            }

            // Key method of the Subject
            @Override
            public void notifyObservers() {
                for (Observer observer : observers) {
                    observer.update(); // The Model
                       requests EVERY View to update
                       itself
                }
            }

            // Business logic method, modified to notify
            public void setEmail(String newEmail) {
                this.email = newEmail;
                // AFTER modifying the state, the Model
                   notifies autonomously.
                notifyObservers(); // <--- No dependency
                   on the Controller!
            }

```

```
36              // Getters...
37              public String getNome() { return nome; }
38              public String getEmail() { return email; }
39          }
```

```
1           // view
2           public class UserView implements Observer {
3               private UserModel model; // Reference to
                    Model to fetch data
4               private UserController controller; //
                    Reference to Controller to send input
5
6               public UserView(UserModel model,
                    UserController controller) {
7                   this.model = model;
8                   this.controller = controller;
9                   // Crucial step: the View registers as an
                        observer of the Model
10                  this.model.registerObserver(this);
11              }
12
13              // Method called by the Model when state
                    changes (the notification)
14              @Override
15              public void update() {
16                  // The View reacts to notification by
                        calling Model getters
17                  String nome = model.getNome();
18                  String email = model.getEmail();
19
20                  System.out.println("--- View Aggiornata (
                        Notifica Ricevuta) ---");
21                  System.out.println("Nome: " + nome);
22                  System.out.println("Email NUOVA: " +
                        email);
23                  System.out.println("
                        ----------------------------------------
                        ");
24              }
25
26              // Method to send input to the Controller
27              public void fireEmailChange(String newEmail)
                    {
```

```
28                  // The View talks ONLY with the
                        Controller
29                  controller.handleEmailChangeRequest(
                        newEmail);
30              }
31          }
```

```
1           // controller
2           public class UserController {
3               private UserModel model;
4               // The View is used only to be passed to the
                    Model,
5               // but its presence is no longer needed for
                    updating.
6
7               public UserController(UserModel model) {
8                   this.model = model;
9               }
10
11              // Deals only with input: takes request and
                    passes it to the Model.
12              public void handleEmailChangeRequest(String
                    newEmail) {
13                  System.out.println("Controller: Ricevuta
                        richiesta cambio email.");
14
15                  // The Controller DOES NOT call the View.
16                  // Its sole responsibility is UPDATING
                        THE MODEL.
17                  model.setEmail(newEmail);
18
19                  // The Model will do the rest thanks to
                        the Observer pattern.
20              }
21          }
```

Advantages:

- Clear separation between interface, data, and logic.
- Reuse of Views with different Models.
- Independence from representation details.

Involved Patterns:

- **Observer:** Views are Observers of the Model (pull).
- **Observer (GUI events):** Controllers observe View events.
- **Strategy:** Controllers act as Strategies for Views (input management).
- **Composite:** Views are often composed of multiple GUI components.

Notes:

- The Controller acts as an *Adapter* between Model and View.
- Circular structure: View→Controller→Model→View.

Disadvantages:

- Circular dependency → difficult testing and incremental development.
- GUI testing is complex (different libraries, asynchronous events).
- View and Controller depend on the graphic library (e.g., JavaFX) → low portability.

Typical Use: development of **interactive GUIs**. **Limitation:** complicated testing and strong dependency on graphical interfaces.

7.13 Model View Presenter (MVP)

Purpose: eliminate direct dependency between Model and View, facilitating **independent testing**.

Key Difference: the **Presenter** replaces the Controller as a *complete intermediary* between View and Model.

Flow:

1. The View sends input to the Presenter.
2. The Presenter processes input and updates the Model.
3. The Model notifies the Presenter (no longer the View).
4. The Presenter updates its own View.

```java
// model
public class CounterModel {
    private int count = 0;
    private CountObserver presenter; // Now notifies only the Presenter

    public void setObserver(CountObserver p) {
        this.presenter = p;
    }

    public void increment() {
        count++;
        // The Model notifies the Presenter, NOT the View.
        if (presenter != null) {
            presenter.onCountChanged(count);
        }
    }

    public int getCount() {
        return count;
    }
}
```

```java
// view
public class ConsoleCountView implements CountView {
    private CountPresenter presenter;

    // The View must know its Presenter
    public ConsoleCountView(CountPresenter presenter) {
        this.presenter = presenter;
    }

    // Method called by Presenter to draw on screen
    @Override
    public void displayCount(String countValue) {
        System.out.println("VIEW: Valore attuale del Contatore: " + countValue);
    }
```

```
15
16              // Simulates user action (Click on + button)
17              public void userClickedIncrement() {
18                  // The View immediately delegates the
                        request to the Presenter.
19                  presenter.handleIncrementRequest();
20              }
21          }
```

```
1           // Contract for Presenters observing the Model
2           public interface CountObserver {
3               void onCountChanged(int newCount);
4           }
5           // Presenter
6           public class CountPresenter implements
                CountObserver {
7               private CounterModel model;
8               private CountView view;
9
10              public CountPresenter(CounterModel model,
                    CountView view) {
11                  this.model = model;
12                  this.view = view;
13
14                  // 1. The Presenter registers as an
                        Observer of the Model
15                  this.model.setObserver(this);
16              }
17
18              // Input handling logic (called by the View)
19              public void handleIncrementRequest() {
20                  // 2. The Presenter commands the Model
21                  model.increment();
22                  // No need to call view.displayCount here
                        .
23              }
24
25              // Reaction logic to Model (Observer method)
26              @Override
27              public void onCountChanged(int newCount) {
28                  // 3. The Presenter receives notification
                         from Model
29                  // 4. Presenter decides presentation
```

```
                    format and commands the View
30                  String formattedCount = String.valueOf(
                        newCount); // Presentation logic
31                  view.displayCount(formattedCount);
32              }
33          }
```

Effects:

- Model and View never communicate directly.
- The Presenter becomes a **bidirectional Adapter**.
- Each component can be tested in isolation (Design for Testing).

Warning:

- Avoid **reference escaping** between Model and Presenter (the presenter has no way to modify the model directly except via set methods).
- Maintain strict separation of responsibilities.

Result: a more testable, less interface-dependent, and more modular system.

7.14 Builder

Problem: initializing objects with many parameters (mandatory + optional) in a readable, safe, and consistent manner.

7.14.1 Telescoping Constructor Pattern

Idea: multiple constructors with different combinations of optional parameters. |Problems:|

- Number of constructors grows exponentially.
- Ambiguity with parameters of the same type (cannot have two constructors where one initializes only A and the other only B if both are of the same type).

7.14.2 JavaBeans Pattern

Idea: constructor with mandatory parameters only + setters for optional ones. |Problems:|

- Objects are not immutable (requires setters → fields cannot be final).
- Possible inconsistent intermediate states.

7.14.3 Builder Pattern

Idea: combine the advantages of both:

- Inner class `Builder` containing identical fields as the outer class.
- `Builder` has a constructor for mandatory parameters and fluent setters for optional ones.
- The `build()` method creates the final instance.

```
1  public class MyClass {
2      private final T0 optionalField1;
3      private final T1 mandatoryField;
4      private final T2 optionalField2;
5
6      private MyClass(Builder builder) {
7          mandatoryField = builder.mandatoryField;
8          optionalField1 = builder.optionalField1;
9          optionalField2 = builder.optionalField2;
10     }
11
12     public static class Builder {
13         private T1 mandatoryField;
14         private T0 optionalField1 = defaultValue1
               ;
15         private T2 optionalField2 = defaultValue2
               ;
16
17         public Builder(T1 mf) {
18             mandatoryField = mf;
19         }
20
21         public Builder withOptionalField1(T0 of)
               {
22             optionalField1 = of;
23             return this;
24         }
25
26         public Builder withOptionalField2(T2 of)
               {
27             optionalField2 = of;
28             return this;
29         }
```

```
30
31                  public MyClass build() {
32                      return new MyClass(this);
33                  }
34              }
35          }
```

Advantages:

- **Immutable** objects (final fields).
- No inconsistent intermediate states.
- Fluent and readable notation.
- Thread-safe and secure construction.

Note: the nested `Builder` class is **static**, so it can be instantiated without an outer object.

Typical syntax (note that the various builder methods return themselves, allowing method chaining):

```
1   MyClass x = new MyClass.Builder(mandatory)
2   .withOpt1(o1)
3   .withOpt2(o2)
4   .build();
```

Conclusion: creational pattern that enables modular, secure, and readable construction of complex objects.

8 UML

a language whose purpose is to create standards for the visual representation of software

8.1 Natural Text Analysis

8.1.1 Design Organization

there are three main ways to begin the workflow

- **Patterns:** recognize common situations starting from data.
- **TDD (Test Driven Design):** start from the simplest solution, defining classes only as needed.
- **Noun Extraction:** extracting nouns from specifications to identify classes and relationships.

8.1.2 Noun Extraction

to identify them:

1. Identify nouns/noun phrases in the specifications.
2. Filter based on criteria (redundancy, vagueness, events, metalanguage, external concepts, attributes).
3. Identify relations and build class hierarchies.

8.1.3 Filtering Criteria

- **Redundancy:** synonyms or equivalent expressions (*library member* vs *member of library*).
- **Vagueness:** generic names (e.g., /items|).
- **Events/operations:** do not represent states (e.g., /loan|).
- **Metalanguage:** logical elements of the program (*system*, *rules*).
- |External:| concepts outside system control (*library*, *week*).
- **Attributes:** primitive data (*name of member*).

8.1.4 Class Relationships

- Connect classes with lines (not arrows), then define UML cardinalities.
- Look for generalizations and factorizations:
  - *StaffMember* is a type of *LibraryMember*.
  - *Item* generalizes *CopyOfBook* and *Journal*.
- Constraints can be expressed with OCL (Object Constraint Language) (certain details cannot be depicted graphically and require other methods).

8.1.5 State: Concrete vs Abstract

it is also necessary to ensure that the class state is represented optimally:

- **Concrete State:** combination of values of all attributes (each individual book with its properties (title, author, code, position...)).
- **Abstract State:** compact and meaningful representation of concrete states (general logical condition (if the code starts with 'L', the book is "read-only in the hall")). An abstract state encapsulates multiple concrete states and is a concept (e.g., temp from 0.000001 to 18 degrees: cold).

8.1.6 Special Cases

- **Stateless object:** with neither attributes nor state (objects representing functional abstractions).
- **Immutable objects:** only one possible state (e.g., *String* object).

8.2 State Diagram

8.2.1 Concept and Structure

- Derived from /State Charts|.
- Automaton: $\langle S, I, U; \delta, t, s_0 \rangle$.
  - S: finite, non-empty set of states;
  - I: finite set of possible inputs;
  - U: finite set of possible outputs;

  - δ: transition function;
  - t: output function;
  - s0: initial state.

- $\delta : S \times I \to S$ (transition), s_0 initial state.

8.2.2 Example: Moore Flip-Flop

- States: 0 and 1.
- Inputs: S (Set), R (Reset).
- Output: current state.

8.2.3 Mealy Automata

$$t : S \times I \to U$$

The output depends on the state and the input, whereas in Moore automata, the state depends on S and I; this changes things because if we remained in the same state, Moore would yield the same output, but Mealy would not.

8.3 In UML

to represent these diagrams in UML:

- Each state: rectangle; initial state: black dot.
- Excessive complexity $\Rightarrow$ too many responsibilities $\Rightarrow$ decompose the class.

8.3.1 Actions and Events

- **Event:** method triggering a transition.
- **Action:** resulting effect (invocation on other objects).

8.3.2 Guards

- Disambiguate transitions from the same state (a book being borrowed; the system could move to the unavailable state if it were the last copy, otherwise it would remain in the available state).

8.3.3 Special Events

- **Time event:** timed events, after(duration) (indicates maximum residence time in a state).
- **Change event:** when(condition) (a sort of trigger, events sparked by a change).

8.4 Superstate

- Groups common states, eliminating redundancies (a traffic light is either off or on; if on, it turns into another state diagram).
- **History state:** we can also have a diagram that remembers the previous state (e.g., does a traffic light turn on green? useful for intersections).
- **Region:** can also represent concurrency (e.g., write something every 5 inputs) (concurrent states separated by dashed lines).

8.5 Class Diagram

8.5.1 Concept and Structure

- Static view of the software: classes, methods, attributes, relationships.
- **objects:** Class, **I:** Interface, **A:** Abstract, |E:| Enum.
- **methods:** preceded by a circle and return value type.
- |attributes:| preceded by a square.
- **relationships:** arrows connecting objects.

8.5.2 Graphical Rules

- methods and attributes can be private, protected, or public (red, yellow, green).
- Italics: abstract element.
- Underlined: static.
- « »: stereotype (label to identify an object).

8.5.3 UML Relationships

- **Dependency (dashed):** A depends on B.
- **Association (line):** bidirectional link (without arrow) or monodirectional (with arrow).
- **Aggregation (white diamond):** "has a" — a stronger relationship than association (both students and professor remain as such even if not associated with each other, but here the link is tighter).
- **Composition (black diamond):** lifespan tied to container (The contained object cannot be born before the container is born and cannot die after the container dies).
- **Implementation (triangle):** a class can implement an interface.

8.6 Sequence Diagram

8.6.1 Concept and Structure

- Represents the temporal flow of invocations.
- Elements: actors, invocations, returns, loops (/loop|), conditions (/opt|).

8.7 Activity Diagram

8.7.1 Concept and Structure

- States → activities.
- Implicit transitions.
- similar to state diagrams.

8.7.2 Abstraction Levels

- Internal logic of processes.
- Flow of a method.
- Details of a use case.

8.7.3 Synchronization

- JOIN/FORK bars (AND or OR).
- Mandatory start and end.

8.7.4 Decisions

- Mutually exclusive edges.
- Complete coverage of conditions.
- Differ from guards (external decision, not within the system).

8.7.5 Swim Lanes

- Division by responsibility (vertical lanes).

8.8 Use Case Diagram

8.8.1 Concept and Structure

- Represent actor–system interactions.
- Simple language, client-oriented.

8.8.2 Components

- Actors, Use Cases, Relationships.
- Each actor $\leftrightarrow$ at least one use case.

8.8.3 Relationships

- **Include:** factoring out commonalities.
- **Extend:** exceptional behavior.
- **Generalization:** inheritance between actors or use cases.

8.9 Component Diagram

8.9.1 Concept and Structure

- Shows and groups system components.
- Components = rectangles, interfaces = circles.
- Stereotypes: labels enclosed between « ».

8.9.2 Component Identification

- Replaceable/versionable parts.
- Well-determined functions.
- Hierarchical structure.

8.10 Deployment Diagram

- Shows the physical distribution of resources at runtime.
- Nodes = physical machines; connections = protocols (HTTP, RMI, etc.).
- Useful to optimize deployment and resource distribution.

UML Diagrams Summary Table

Diagram	Focus and Objective	Questions It Answers
Use Case	Shows functional system requirements: **what** the system does relative to external actors.	Who uses the system? What are the main features provided?
Activity	Models the |logical flow| (sequence of actions, decisions, loops) to complete a process or specific use case.	How does a process unfold from start to finish? What are the possible decisions and branchings?
Sequence	Models the |chronological order| of messages exchanged between objects to achieve a goal.	In what sequence do messages pass between objects? What is the chronological order of events?
Component	Models the organization and dependencies between software components (modules, libraries, services) of the system.	How is the code split at a logical level? Which components depend on others?
Deployment	Models the |physical arrangement| of software components on hardware nodes (servers, devices) during execution.	On which servers or physical devices does each component execute? How are hardware nodes connected?

9 Mocking

9.0.1 Nomenclature:

- **SUT:** System Under Test.
- **DOC:** Dependent-On Component.

9.1 Structure of a Test

Each test must be clear and readable, following 4 phases:

1. **Set Up:** initializing the SUT and Test Doubles.
2. |Exercise:| executing the code to be tested.
3. **Verify:** checking results (assertions).
4. **Teardown:** cleaning up and restoring the environment.

Each test must verify a single functionality to increase fault localization.

9.2 Test Double

Generic term identifying any object that replaces a real component (DOC) during testing.
Used to:

- simulate components that are unavailable or have side effects;
- control SUT inputs/outputs;
- force complex or random scenarios.

Main types:

- **Dummy:** passed as a parameter but never used.
- **Stub:** provides predefined answers to indirect inputs.
- **Mock:** acts like a stub but additionally tracks which methods the SUT called and with what values, to verify that SUT behavior is correct.
- **Spy:** unlike mocks, spies stem from a real object, creating a spy object from it to observe calls made by the SUT.
- **Fake:** a real but simplified implementation (e.g., in-memory DB).

9.3 Rules for Mocking

- Only one real object per test (single `new`).
- All DOCs must be mocked (no real components slowing things down).
- Do not mock standard library classes (e.g., `List`).
- Possible to execute partial **spy**ing on a real object.

9.4 Mockito

Open-source framework for creating and managing Test Doubles. Based on Reflection, it improves readability and control.

Creating Test Doubles

- `mock(Class.class)`: creates a dummy, stub, or mock with minimal implementation.
- `spy(obj)`: creates a spy from a real object (calls real methods if not stubbed).

Stubbing Defines the behavior of mocked methods.

- `when(mock.method(args)).thenReturn(value)`
  states that when that method is called, it must always return value.
- `thenThrow()`, `thenAnswer()`, `thenCallRealMethod()`
  same approach, must return an exception/a response, etc.
- `doReturn()/doThrow()/doNothing()`
  used for `void` methods.

Verifying Calls `verify(mock, howMany).method(args)`, often one wishes to verify not just return values but how many times a method was called:

- `times(n)`, `never()`, `atLeast(n)`, `atMost(n)`
- `InOrder inO = inOrder(mock1, mock2,...)` to verify order.

example of verifying that method doSomething was called with a specific object (sometimes complex, so argumentCaptor is used)

Capturing Parameters

```
ArgumentCaptor<Person> arg = ArgumentCaptor.
    forClass(Person.class);
verify(mock).doSomething(arg.capture()); // traps
     the input object inside arg
assertEquals("John", arg.getValue().getName());
    // verifies the passed object

// verify is used instead of when because we are
    not defining a behavior but verifying it
```

Argument Matchers Allow avoiding specification of exact values:

- `any()`, `anyInt()`, `anyString()`
- `eq(value)` for specific equality.
- `argThat(...)` for custom conditions.

Reset `reset(mock)` restores the original state of the Test Double.

9.5 Example: Observer Pattern (Pull)

```
@Test
void modelTest() {
    Model model = new Model();
    Observer obs = mock(Observer.class);
    Observer obs1 = mock(Observer.class);
    model.addObserver(obs);
    model.addObserver(obs1);
    model.setTemp(42.0, scale);
    verify(obs).update(eq(model), eq("42.0"));
    verify(obs1).update(eq(model), eq("42.0"));
}
```

9.6 Injection with Mockito

mock objects can be injected into real objects, provided we place the `@ExtendWith(MockitoExtens` notation at the top.

```
@ExtendWith(MockitoExtension.class)
class TestPartita {
    @Mock Tavolo mockedTable;
    @InjectMocks Partita SUT;

    @Test
    void test() {
        when(mockedTable.inMostra(Card.get(...)))
            .thenReturn(true);
        assertThat(SUT.
            controllaSeCartaPresenteSuTavolo(...))
            .isTrue();
    }
}
```

Rules:

- @Mock: mocked objects.
- @InjectMocks: real SUT into which mocks are injected.
- Injection via constructor, setter, or directly into private attributes if they don't exist (provided it's not final), e.g., inject of table into Partita.
- `final` attributes are not modifiable ⇒ use **Mocked Construction**.

9.7 Mocked Construction

Allows mocking objects at the moment of their creation (a lambda function must be passed to try as a parameter to avoid inconsistent states; indeed, if I didn't use the lambda, I would have to invoke the mocked object to define its behaviors, causing inconsistency).

```
@Test
void constructorTest() {
    try (MockedConstruction<MultiMazzo> mc =
    mockConstruction(MultiMazzo.class, (mock, ctx
        ) -> {
        when(mock.draw()).thenReturn(Card.get("2C
            "));
```

```
6               })) {
7                   Mazziere m = new Mazziere();
8                   assertThat(m.draw()).isEqualTo("2C");
9               }
10          }
```

- Mocks every `new MultiMazzo()`.
- Allows stubbing at creation time.
- Verifications via `mocked.constructed().get(0)`.

9.8 Mocking of Iterable

if this implementation were used, the first time it is called it returns the correct element; from the second time onwards, the iterator must be recreated (in Java, iterators are throw-away) and it would therefore fail.

with thenReturn → always return the same iterator → works only the first time;

with thenAnswer → recreated at each call, solving the problem.

```
1           Iterable<Integer> mockIterable = mock(Iterable.
               class);
2           when(mockIterable.iterator()).thenReturn(List.of
               (1, 2, 3).iterator());
```

Used for objects implementing `Iterable`.

```
1           public static <T> void whenIterated(Iterable<T> p
               , T... d) {
2               when(p.iterator()).thenAnswer(inv -> List.of(
                   d).iterator());
3           }
```

Example:

```
1           MockUtils.whenIterated(partita, ritorno[]); //
               partita is the mock on which behavior is
               defined, ritorno are the objects over which it
                iterates
```

Guarantees that each iteration creates a new valid iterator.

—

9.9 Test Double Types Summary

Type	Description
Dummy	Placeholder only, never used.
Stub	Returns predefined values.
Mock	Verifies indirect interactions/outputs.
Spy	Monitors calls on a real object.
Fake	Simplified non-production implementation.

10 Verification and Validation

10.1 Verification and Validation

10.1.1 Terminology

- **Verification**: evaluating software against **formal specifications** (analyst).
- **Validation**: evaluating software against |informal requirements| (client).
- **Main differences**:
  - Requirements: language close to the client's domain (informal).
  - Specifications: formal language close to the developer.
  - Requirements can change, specifications are more stable.
- Verification ⇒ testing specifications (simpler to write).
- Validation ⇒ testing requirements (more complex to formalize).
- Common objective: detect errors.

IEEE Terminology :
we must strictly define 3 terms:

10.1.2 |

Failure|

- External observable deviation from correct behavior.
- Not visible in the code itself, but in behavior.
- Can concern specifications (verification) or requirements (validation).
- **Example**:

```
1              static int raddoppia(int par) {
2                  int risultato;
3                  risultato = (par * par);
4                  return risultato;
5              }
6              // Input: 3, Output: 9 implies a failure
```

10.1.3 Fault/Anomaly

- The spot in code causing a failure.
- **Necessary but not sufficient condition** to cause a failure.
- Example: in the previous example, the fault is `risultato = (par * par);`.
- Does not always cause failure (e.g., arguments 0 or 2).
- Possible defect masking (errors neutralizing each other).

10.1.4 Mistake

- Cause of a fault → human error.
- Types:
  - Typo (`*` instead of `+`).
  - Conceptual error (not understanding "doubling").
  - Language error (confusing operators).

10.1.5 Ariane 5 Case

- **Date:** June 4, 1996.
- **Event:** rocket explosion due to software failure.
- **Failure:** overflow during `float 64` → `int 16` conversion.
- **Fault:** missing handling of the overflow exception.
- **Mistake:** reusing code from Ariane 4, where this overflow could not occur.
- **Lesson:** always analyze the validity limits of software across different contexts.

10.2 |

Verification and Validation Techniques|

10.2.1 General Classification

- **Static techniques:** syntactic code analysis.
  - Examples: formal methods, dataflow analysis, statistical models.
- **Dynamic techniques:** analysis via execution.
  - Examples: testing, debugging.
- Static = faster and more comprehensive, but harder to define.
- Dynamic = easier to create, but cover finite program states.

10.2.2 Conceptual Pyramid

- **Ideal vertex:** perfect verification (logical proof or exhaustive search).
- **Pyramid bases:**
  - **Excessive simplification of goals** (left): if you want to verify that a pointer points to the right object, it is not enough to verify it is not null.
  - **Pessimistic inaccuracy** (center): "If I cannot prove the absence of a problem, I assume it exists."
  - **Optimistic inaccuracy** (right): "If I find no problems, I assume none exist."
- Real techniques fall between these extremes.

10.2.3 Formal Methods

- Purpose: prove the absence of faults in the final product (mathematically prove that the program can never fail).
- Examples:
  - Dataflow analysis.
  - Logic correctness proof.
- Follows the pessimistic inaccuracy approach.

10.2.4 Testing

- Purpose: detect failures (execute the program and verify it behaves correctly on many significant cases).
- Increases confidence in reliability (does not prove correctness).
- Types:
  - **White box:** complete access to code.
  - **Black box:** testing interfaces without viewing code.
  - **Gray box:** partial knowledge (e.g., MVC pattern).
- Follows optimistic inaccuracy logic.
- **TDD:** writing tests before code $\Rightarrow$ black box approach.

10.2.5 Debugging

- Purpose: locate the fault causing a known failure.
- Requires a reproducible failure.
- **Differences with testing:**
  - Testing: searches for failures.
  - Debugging: identifies the cause.
- Approaches:
  1. Step-by-step analysis of intermediate states.
  2. Divide and conquer: sectional analysis with breakpoints or print statements.

10.3 Testing

10.3.1 Definitions

10.4 Legend

:

- $D \rightarrow$ domain
- $R \rightarrow$ codomain

- $P(d) \rightarrow$ execution of a program on a certain input d belonging to D
- $ok(P, D) \rightarrow$ P is correct on any d of D

- **Correct Program:**

$$P \text{ is correct} \Leftrightarrow \forall d \in D,\ ok(P, d)$$

Definition of Test

- Test $T \subseteq D$, test case $t \in T$.
- Execution: $\forall t \in T$, execute $P(t)$ and compare with expected result.
- P passes T if:

$$ok(P, T) \Leftrightarrow \forall t \in T,\ ok(P, t)$$

- A test succeeds if it detects failures:

$$successo(T, P) \Leftrightarrow \exists t \in T : \neg ok(P, t)$$

Ideal Test

- T is ideal for P if:

$$ok(P, T) \Rightarrow ok(P, D)$$

- **Dijkstra's Thesis:** testing can show the presence of bugs but never prove their absence.
- Exhaustive testing $T = D$ is impractical (e.g., 2^{64} combinations $\Rightarrow$ >600 years at 1ns/test).

10.4.1 Properties

$C \rightarrow$ criterion used to choose which tests to perform

Reliability :
a criterion is reliable if all tests conducted on P either all succeed or none do:

$$affidabile(C, P) \Leftrightarrow (\forall T_1, T_2 \in C)\ [successo(T_1, P) \Leftrightarrow successo(T_2, P)]$$

Validity :
a criterion is valid if, whenever P is incorrect, there exists at least one test that succeeds (identifies the fault):

$$valido(C, P) \Leftrightarrow (\neg ok(P, D) \Rightarrow \exists T \in C \mid successo(T, P))$$

Example

```
1  static int raddoppia(int par) {
2      int risultato;
3      risultato = (par * par);
4      return risultato;
5  }
```

- $T = \{0, 2\}$: reliable but not valid.
- $T = \{0, 1, 2, 3, 4\}$: valid but not reliable.
- $T = \{d > 18\}$: valid and reliable (tests all succeed and the program always fails).

Conclusion

- Valid $\wedge$ reliable criterion $\Rightarrow$ ideal test on any program (impossible according to Dijkstra).
- Formally:

$$affidabile(C, P) \wedge valido(C, P) \wedge T \in C \wedge \neg successo(T, P) \Rightarrow ok(P, D)$$

10.4.2 Usefulness of a Test

the compromise lies in identifying useful tests:

- A test case is useful if it:
  1. Executes the statement containing the fault.
  2. Triggers an incorrect/inconsistent state.
  3. Propagates the inconsistency to the output.
- Selection criterion $\Rightarrow$ how many/which test cases to include.
- Each criterion features a **coverage metric** $\Rightarrow$ measures test goodness.

10.4.3 Selection Criteria

- Objective: approximate an ideal test; each criterion attempts to maximize program coverage.
- Main types:
  1. Statement coverage
  2. Decision coverage
  3. Condition coverage
  4. Decision+condition coverage
  5. Multiple condition coverage
  6. Modified condition/decision coverage (MC/DC)

```
01  void main(){
02      float x, y;
03      read(x);
04      read(y);
05      if (x != 0)
06          x = x + 10;
07      y = y / x;
08      write(x);
09      write(y);
10  }
```

Statement Coverage

- statement = logical unit (no need for all paths of an if to be traversed, it suffices that the if is traversed at least once), assignment is also a statement.
- Every executable statement must be executed at least once.
- Metric: $\frac{\text{executed statements}}{\text{total statements}}$.
- Does not guarantee defect detection (e.g., division by 0 remains undetected).

| Decision Coverage|

- Each decision evaluates to both true and false by at least one test case (i.e., traversing all feasible paths).
- Metric: $\frac{\text{decisions evaluated T/F}}{\text{total decisions}}$.
- **Implies** statement coverage.

| Condition Coverage|

- Each atomic condition evaluates to both true and false.
- Does not imply any of the previous criteria.
- Composed decision: individual conditions can be covered without exploring all branches.
- e.g.: x == 0 and y == 2 (2 tests: one satisfies x == 0, one satisfies y == 2, but in some cases fails to cover all feasible paths).

| Decision and Condition Coverage|

- Each decision and each atomic condition evaluates to both true and false.
- **Implies** decision and condition coverage ⇒ statements.
- e.g.: x == 0 and y == 2 (2 tests: first false, second true; first true, second false; in both tests, despite meeting the criterion, the outcome is false).

Multiple Condition Coverage

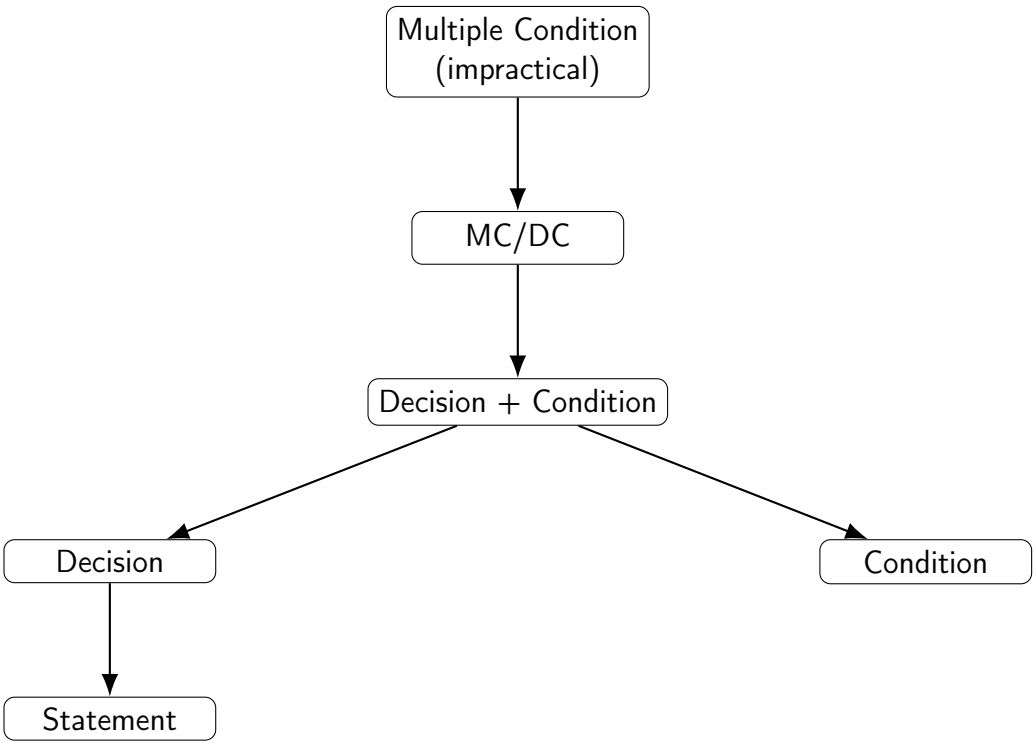
- All T/F combinations of basic conditions.
- Combinatorial growth (2^n combinations).
- **Implies all** previous criteria but is impractical.

| Modified Condition/Decision Coverage (MC/DC)|

- Each basic condition must independently affect the decision's outcome.
- Only $N + 1$ test cases are needed for N basic conditions.
- **Implies** decision+condition coverage.

```
1              Test  A  B  C    Decision Result
2              t1    v  v  v        v true base case
3              t2    f  v  v        f influence of a
4              t3    v  f  v        f influence of b
5              t4    v  v  f        f influence of c
```

10.4.4 Implications Between Coverage Criteria



10.4.5 Other Criteria (Loops)

- Previous criteria do not consider iterations.
- Defects often surface after multiple iterations.
- Criteria testing loops are necessary.

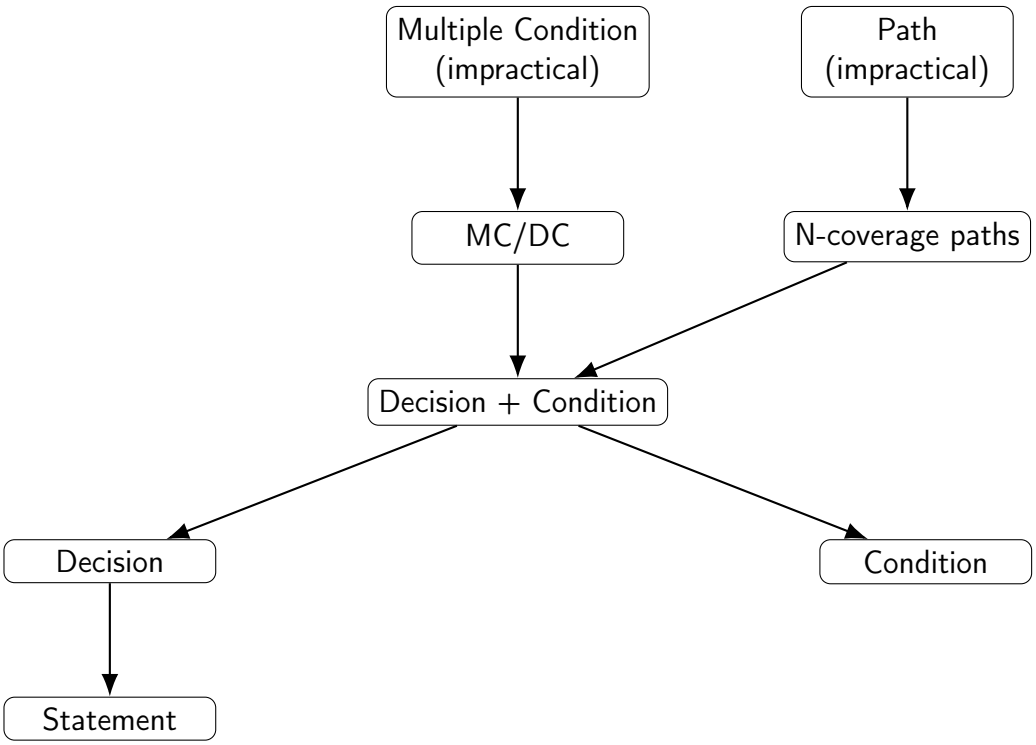
Path Coverage

- Every path through the control graph must be executed at least once.
- Impractical with loops (∞ paths).

n-Coverage of Loops

- Every path containing $\leq n$ iterations per loop must be executed.
- n index of loop with **n=2** is optimal (tests: 0, 1, >1 iterations) (boundary cases and intermediate case).
- **Applicable to real programs**.

10.4.6 Final Map of Implications



11 Static Analysis

11.1 General Definition

- Does not execute the program: analyzes the **source code** (finite set of statements).
- Contrast to **dynamic analysis**, which considers execution states (infinite set).
- Cheaper than testing; does not detect errors related to variable values at runtime.
- Can pinpoint conceptual errors and potential faults.

11.2 Core Tools

1. **Compilers**
2. **Data Flow Analysis**
3. **Static Analysis Guided Testing**
4. **Coverage Criteria**

11.3 |

Compilers| **Perform preliminary static analysis** prior to translation:

- **Lexical analysis:** identifies invalid tokens.

```
dar ciao = 12; // invalid 'dar' keyword
```

- **Syntactic analysis:** checks grammatical structure (the order of words, not their correctness, checking if they make sense together in that manner).

```
int ciao = 12 // missing semicolon
```

- **Type checking:** verifies type compatibility.

```
int ciao = 12;
ciao += "hola"; // incorrect type
```

- **Data flow analysis:** finds unreachable code or inconsistent values.

```
1              if 1 != 1 {
2                  int a = 1; // unreachable
3              }
```

11.4 Data Flow Analysis

Objective: study how variables are defined, used, and cleared in the code. Each operation belongs to one of three types:

d: definition (assignment or modification)
u: use (reading the value)
a: voidance / cancellation (value no longer valid)

Core Rules:

1. Every use (**u**) must be preceded by a definition (**d**) with no intervening voidances. ⇒ Sequence **a-u** = error.
2. Every definition must be followed by at least one use before a new d or a. ⇒ Sequences **d-a**, **d-d** = error.
3. Every voidance must be followed by a definition before a new use. ⇒ Sequence **a-a** = error.

11.5 Practical Example

```
1 void swap(int &x1, int &x2) {
 2      int x1;
 3      x3 = x1;
 4      x3 = x2;
 5      x2 = x1;
}
```

Variable	Sequence	Anomalies
x1	a u u a	use without definition
x2	d u d	none
x3	d d d	multiple definitions without use

x1: (line 2 clears the assignment made in input (line 1), 3,5 use, exit triggers cancellation)

11.6 Sequences and Paths

A **sequence** of a variable a according to path p is:

$$P(p, a) = \text{concatenation of operations on } a$$

Example:

```
01  void main() {
02      float a, b, x, y;
03      read(x);
04      read(y);
05      a = x;
06      b = y;
07      while (a != b)
08          if (a > b)
09              a = a - b;
10          else
11              b = b - a;
12      write(a);
13  }
```

P(([1,2,3,4,5,6,7,8,9,7,12,13]),a) = a2,d5,u7,u8,u9,d9,u7,u12,a13 (11 is skipped since it resides in the else branch and is omitted in this path)

11.7 Regular Expressions

Program paths can be represented as regular expressions:

- The | text symbol represents **decisions**.
- The * symbol represents **loops**.

(SEE IMAGE IN PROFESSOR'S NOTES)
Limitation: a regular expression is a **pessimistic abstraction** → includes non-feasible paths.

11.8 |

Testing and Static Analysis|

- Static analysis aids test case selection.
- Criteria are based on **definition–use (DU) sequences** (recommends testing each definition up to its use).

11.9 Terminology

$$\text{def(i)} = \text{set of variables defined in node } i$$

$du(x,i) = \{j \mid x \in def(i),\ x \text{ used in } j, \text{ path } i \to j \text{ free of new definitions of } x\}$

(du(x,i) is the set of nodes j where x is defined in i and used in j, etc.)

11.10 Derived Coverage Criteria

```
01  void main() {
02      float a, b, x, y;
03      read(x);
04      read(y);
05      a = x;
06      b = y;
07      while (a != b)
08          if (a > b)
09              a = a - b;
10          else
11              b = b - a;
12      write(a);
13  }
```

du(5,a) = 7,8,9,11,12 and du(9,a) = 7,8,9,11,12

1. **Definition Coverage (Cdef)**: every definition tested up to **at least one use** (free of intermediate definitions, hence each path must have a d, then a u before another d).
   a is indeed defined in 5 and 9 and we must find tests covering both:
   input: <8,4>

2. **Use Coverage (Cuse)**: every definition tested up to **all uses**.
   here we require 3 tests:
   input <4,8> <12,8> <12,4>

   - the first covers only uses for a defined in 5 (d5) of while and else (u7,u11,u12).
   - the second only uses of a defined in 9 (d9) of while (u7,u9,u11,u12). This tests if a remains larger than b even after subtraction.
   - the third serves to evaluate how the value of a modified in the loop (d9) reaches final printing (u12) passing through a cycle where it is untouched.

3. **DU Path Coverage (CpathDU)**: every path from every definition to every use tested (impractical due to combinatorics).

11.11 Beyond Variables

Extension to generic objects (e.g., files):

a: open (read/write)
l: read
s: write
c: close

Typical Rules:

- l, s, c must be preceded by a with no intermediate closures.
- a must be followed by c before a new opening.

11.12 |

Budget Coverage Criterion|

- Test cases are created until resources (time/money) run out.
- No guarantee of efficacy → criterion **not recommended**.

12 Review Techniques

12.1 Introduction

- Automated tools (testing and code analysis) do not detect errors stemming from specifications misunderstandings.
- Human intervention required via code **reviews**.
- Often performed by an independent testing team → more impartiality and higher efficacy.

12.2 Main Techniques

1. Bebugging
2. Mutation Analysis
3. Object Oriented Testing
4. Functional Testing
5. Software Inspection
6. Statistical Models
7. Debugging

12.3 |

Bebugging|

- Developers intentionally insert n bugs into the code.
- The testing team is informed of the number n of bugs to find.
- Objective: stimulate searching and evaluate testing efficacy.
- Metric: percentage of discovered bugs/n.
- Limitation: difficulty calibrating artificial bug difficulty.

12.4 Mutation Analysis

- Evolution of bebugging.
- Given a program P and a test T, mutants Π (variants of P) are generated.
- T is executed on each mutant:
  - If P is correct, mutants must produce different results.
  - Measures the test's ability to detect differences, not the correctness of P.

12.4.1 Mutant Coverage Criterion

- T satisfies the criterion if each mutant $\pi \in \Pi$ produces at least one output differing from P.
- Metric: fraction of detected mutants relative to total (if a mutant goes undetected, tests must be added to capture it).

12.4.2 Mutant Generation

- Mutants $\rightarrow$ minimal differences compared to P.
- Limited number (hundreds/thousands) due to time constraints.
- Mutant quality determined by **mutant operators**.

Mutant Operators

- Modifications on:
  - Constants/variables (e.g., value swapping).
  - Operators/expressions (e.g., $< \rightarrow \leq$).
  - Statements (e.g., while $\rightarrow$ if).
- Domain-specific:
  - Concurrent (synchronization).
  - Object-Oriented (interfaces and methods).
- Automated tools often operate on the executable since generating mutants can be computationally expensive (less overhead).

12.4.3 Higher Order Mutation (HOM)

- Applies multiple changes to already mutated code (2 errors can mask each other).
- Purpose: create mutants that are harder to detect → more thorough testing.

12.4.4 Automation

- The goal is not checking correctness but testing **depth**.
- Correctness oracle is not required (since knowing program inner mechanics is unnecessary) → higher automated capability.
- Cost: $n + 1$ executions for n mutants ($+1$ for the real program).
- Problems:
  - Repetition of random tests (to generate the test until the mutant is covered).
  - Equivalent mutants (identical semantics).
  - Timeouts required to prevent infinite loops.

12.5 |

Object-Oriented Testing|

- In OO languages, the testable unit is the **class**, not the function.
- Methods depend on the object's state.

12.5.1 Inheritance and Dynamic Binding

- Inherited methods must be re-tested within subclasses.
- Testing is not inherited, but **test cases** can be.
- Dynamic binding complicates coverage (methods resolved at runtime).

12.5.2 |

Testing a Class|

- Isolate the class using *stubs*.
- Implement missing methods.
- Add functions to access state.
- Use a *driver* class to test methods according to a coverage criterion.

12.5.3 Coverage Criteria for Classes

- Based on a |state machine|:
  1. Node coverage.
  2. Edge coverage.
  3. Input/output edge pairs coverage (if I have 2 incoming and 2 outgoing edges, I need 4 tests, one for each combination).
  4. All paths coverage (often infinite, impractical).

Type of Testing

- If the state machine is part of specifications → **black box**.
- If derived from code → **white box**.

12.6 Functional Testing

- Verifies external program behavior (black box).
- Based on **specifications**, not implementation.
- Identifies errors such as:
  - Missing functionalities.
  - Interface or performance errors.

12.6.1 Core Techniques

1. Graph-based (sequence diagrams, etc.).
2. Partitioning into equivalence classes (splitting domain values causing the same behavior to test distinct behaviors).
3. Boundary value analysis at partition borders (frontier testing).
4. Comparison testing (New V vs Old V searching to avoid regression).

12.6.2 Interface Testing

- Verifies communication between components.
- Interface types:
  - Parameter-based, shared memory, synchronous methods, message-based.
- Common errors: incorrect parameters, wrong preconditions, synchronization issues.

12.6.3 Equivalence Classes

- Input domains split into sets of values producing the same behavior.
- Types:
  - Valid inputs.
  - Invalid inputs.
- Objective: test all behavior classes, not all values (e.g., one for valid inputs and one for invalid ones concerning an ATM PIN, not all invalid ones).
- another example is correct values (from 200 to 400); incorrect equivalence classes can be $n < 200$ and $n > 400$.

12.6.4 Boundary Testing

- Values at the boundaries of equivalence classes are tested.
- Errors manifest most frequently at limits.

12.6.5 Category Partition

- Method to derive equivalence classes from specifications.
- Core steps:
  1. Specification analysis and category identification.
  2. Choice of significant values.
  3. Definition of constraints across categories.
  4. Test writing (generally following the 0, 1, 2 pattern, e.g., if the pattern requires a blank space, test with 0, 1, 2 blank spaces).

Alpha and Beta Testing

- **Alpha**: in-house tests (specifications known).
- **Beta**: testing in real environments (diverse configurations).

Constraints

- |Pairwise testing|: testing all pairs of significant values, imposing constraints and logical structures to pick tests.
- Constraint types:
  - *if*: if a property is defined, failing to respect it is useless.
  - *single*: multiple tests performing the same action are redundant (if searching for a word, the specific word changing is irrelevant if no test has more than one occurrence of the searched word).
  - *error*: generates immediate error (missing parameter; obviously it errors out if the parameter is required).

12.6.6 |

Conclusions|

- Difficult to identify all significant cases.
- An oracle is required to know expected results.

13 Object Orientation and Functional Testing

- Functional testing is **black box**: does not depend on implementative details.
- In OO, functional testing does not change at the *unit* level, but **integration testing** does.
- In procedural languages: integration testing uses either **bottom-up** or **top-down** logic.
- In OO: relationships between classes are **non-hierarchical** (associations, aggregations, dependencies), making subsets to be tested hard to isolate.

Useful Tools

- **Use case diagrams** and scenarios → testing involved components.
- **Sequence diagrams** → testing key classes in interactions.
- **State diagrams** → usage analogous to what was previously described.

14 Software Inspection

- Manual verification techniques based on group analysis.
- Example: |pair programming|.
- No tools necessary, high flexibility.
- Can be applied to code, specifications, or executables at any lifecycle stage.

14.1 Fagan Code Inspection

- Method developed by **Michael Fagan (IBM, 1970s)**.
- A group of experts verifies code to isolate errors or inconsistencies.

Core Roles

- **Moderator**: coordinates, selects participants, avoids conflicts of interest.
- **Reader/Tester**: read and interpret the code, search for defects.
- **Author**: answers questions, does not correct during inspection.

Process

1. **Planning**: definition of roles, timelines, meetings.
2. **Overview**: context (initial idea) and materials sharing.
3. **Preparation**: individual understanding of code.
4. |Inspection|: collective analysis, defect identification.
5. **Rework**: correction by the author.
6. **Follow-up**: potential new inspection.

| During the Inspection|

- Defects are logged, **not fixed**.
- Max 2 sessions/day, 2h each, roughly 150 lines/hour.
- Possible "dry run" (manually tracing lines) and use of **checklists**.

14.1.1 Checklists

- Start from common errors and serve to prevent them.
- Can be internal to the company, evolving with the project.
- **NASA Example (1993)**: checklists for C and Fortran split into *functionality, data usage, control, linkage, computation, maintenance, clarity.*

Checklist Item Examples

- Does each module feature a single function?
- Does code adhere to detailed design?
- Are all constants capitalized?
- Few nested #include statements? Sufficient comments?

Incentive Structure

- Discovered defects are **not used to evaluate authors**.
- Readers and testers are incentivized based on found defects.

Variant: Active Design Reviews introduced to prevent reviewers from losing focus

- The author raises questions and interrogates reviewers.
- Increases participant involvement.

14.2 Automation

- Support tools: formatting, electronic checklists, code browsing, team communication, process controls like git merges.

14.3 Pros and Cons

Advantages

- Rigorous, self-improving process.
- Based on experience (checklists).
- Positive social and mental incentives.
- More abstract analysis compared to testing.
- Applicable to incomplete programs.

Disadvantages

- Applicable only at unit level (too many details are hard to grasp).
- Non-incremental: follow-up (new review after correction) is often ineffective.

Fagan's Law (L17)

> Inspections significantly increase productivity, quality, and project stability.

15 Comparison Between Verification and Validation Techniques

- Techniques: structural testing, functional testing, inspection.
- Efficacy varies depending on the project.
- Different techniques find different errors.
- No single technique is always best.

15.1 Combining Techniques

- Applying multiple techniques, even superficially, is more effective than using a single one in depth.

Hetzel-Meyer Law (L20)

> A combination of distinct V/V methods outperforms any single method.

16 Autonomous Testing Groups

Weinberg's Law (L23)

> A developer is unsuited to test their own code.

for this reason, autonomous tester groups are usually preferred

Advantages

- Technical specialization, tool knowledge.
- Detachment from code and psychological independence in evaluation (testing my own code would incentivize me not to find errors).

Disadvantages

- Loss of development skills.
- Greater distance from the project and requirements.
- Shift of responsibility toward testers (a developer is less careful because there is a tester anyway).

Alternatives

- **Staff rotation**: alternating dev/tester roles → less pressure, less specialization, and better balance.
- **Shared staff**: dev = tester (same group doing both) → deep code knowledge but returns to the criticalities seen before.

17 Statistical Models

- Analyze correlations between code metrics (length, indentation, language) and defect count/type.
- Can suggest where to test more, but **do not consider fixes already applied**.

Pareto/Zipf Law (L24)

Roughly 80% of defects originate from 20% of modules (if a zone has an error, it is likely to have more than one).

18 Debugging

- Aims to locate and remove anomalies causing known failures (not to discover failures, tests handle that).
- Relies on failure reproducibility (checking if it truly exists and is not due to specification errors).

| Difficulties|

- Anomaly–failure relationship is not bijective (a failure might stem from multiple anomalies instead of just one).
- Fixes can introduce new errors.

Kernighan Citation

> Debugging is twice as hard as writing the code in the first place.

18.1 Debugging Techniques

- |Naïve|: printing variables → modifies code, low flexibility.
- **Advanced Naïve**: using `#ifdef`, logging, assertions.
- **Memory dump**: complete image of memory (core dump), hard to interpret.
- **Symbolic debugging**: interactive tools, observable variables, /breakpoints|, /watches|, *spy monitors* (do not alter code by adding lines but observe its operation by altering execution (step-by-step), used to see variable values at a given moment).
- **Proof-based debugging**: conditional watches (telling it to check if a var equals another) and dynamic assertions (verifying that certain logical properties remain valid during execution). Encompasses all symbolic debugging features while verifying logical properties.

Additional Debugger Functionalities debuggers can also:

- Modify variables or code at runtime.
- Graphical representations of dynamic data structures.

18.2 Debugging Automation

- **Andreas Zeller, Debugging Book**: introduces potential avenues for a future with automated debuggers
  - **Shrinking input**: automatic reduction of the problematic input.
  - **Differential debugging**: minimal code variations to identify the anomaly.

- **Git bisect**: dichotomous search for the version introducing the bug (compared against the old version).